Introduction To Machine Learning
Prof. Mathieu Salzmann

Arthur Herbette

Tuesday 17 February 2026

Contents

1 Introduction **3**

1.1 Machine Learning Basics . . . 3

Regression vs Classification . . . 4

1.1.1 Unsupervised Learning . . . 4

1.1.2 Supervised Learing . . . 5

1.2 Linear Model . . . 5

1.3 Derivatives and Gradients . . . 10

1.3.1 Interpreting a Linear Model . . . 16

1.3.2 Linear model with multiple outputs . . . 17

1.3.3 Linear Classification . . . 18

1.4 Linear Model for Classification . . . 20

1.4.1 Loss Function ad Empricial Risk . . . 20

1.4.2 Iterative, Gradient-based optimization . . . 26

1.5 Classification Metric . . . 30

1.5.1 From binary to multi-class classification . . . 33

1.5.2 Decision Boundary . . . 34

1.5.3 Multi-class logistic regression . . . 35

Evaluation metrics for multi-class classification . . . 36

1.6 k-nearest Neighbor . . . 38

1.6.1 NN: Algorithm . . . 39

1.6.2 k-nearest neighbors . . . 41

1.6.3 Summary . . . 43

1.7 Nonlinear classification . . . 44

1.7.1 Polynomial curve fitting . . . 45

1.8 Overfitting, Cross-validation and Regularization . . . 48

1.9 Kernel methods . . . 55

1.10 Kernel function . . . 57

1.10.1 Kernelization . . . 58

2 Supervised Linear ML **63**

3 Unsupervised Linear ML **65**

Chapter 1

Introduction

Evaluation

- Final exam: 80% of the grade
- Past year's exams are available on Moodle
- Two graded project milestones
  - Coding exercices build up as a small team project
  - Teams of 3 students: Register the teams by March 22nd
  - Two milestones (tentative: end of Week 8 and end of Week 13)
  - Each is wroth 10% of the final grade

1.1 Machine Learning Basics

What is Machine Learning

Machine learing takes the same form as how a human would learn, by **experience**. The goal is to give our **algorithm**, **data** which would output **insight** about our data.

What data?

Data can be many things, it can be patient information related to births, movie reviews speech (audio file)...

An important thing to highlight is the difference between a **data set** and a **data sample**.

Definition 1. Data sample (or example, or point): is an individual observation

It can be an image, a list of attributes for one patient, etc...

Definition 2. Data set: is a collection of multiple data samples.

Which can be a collection of images, the attributes lists for multiple patients.

What insight?

As said before, the goal of machine learning is to output **insight** about our data. This is insight can be pretty much anything, the category depicted by an image, the influences between different musical genres.

In this course, we will mostly focus on precise insights. In particular, we will study two classes of machine learning problems:

- Regression
- Classification

Regression vs Classification

Regression predicts **continuous value(s)**[1] for a given sample.
This means that our 'guesses' typically follow an **order**. We can then measure "how accurate" our prediction is: predicting 3002g instead of 2977 is better than predicting 2500g. On the other hand, **classification** use 'classes' as prediction, categories do not follow an order: predicting 'tree' instead of 'cat' is just as wrong as predicting 'car'.

Remark 1. We could first see it as in probability/statistic. Regression is the continuous case and classification is the discrete case. Although this makes 'sense', as we will see in the next remark, regression **doesn't mean continuous**.

You are given a dataset of images of paintings. For each painting, you would like to predict the year it was created. Is this a regression or a classification task?
It is a regression task, guessing by one year is better than being off by a complete decade.

1.1.1 Unsupervised Learning

Unsupervised Data

Definition 3. In Unsupervised learning, each sample consists only of an observation, e.g., an image (without information about its content)

What classes of algorithm?

For each learning there will be different classes of algorithms, in this case (unsupervised), the learning relies on unsupervised data. The goal is rather to analyse the observed data set.
What we mean by this is that instead of trying to have direct information about our data (guess the animal on an image), we will more care about how does the data is divided etc... (splitting creating a graph of music genres)

naftaliharris.com

Learning Stage

Learning in this case consists of only one stage: Analyse the whole data, or transform it for further analysis. As said before the goal is not to make 'guesses'. Butto gives more information about our data in order to have better insight.

[1] As said right after, it is not continuous in the sens of a function f in $\mathbb{R}$. But value have an ordering which allows us to have an ordered set of those elements. It's just how I kind of see it

1.1.2 Supervised Learing

Supervised Data

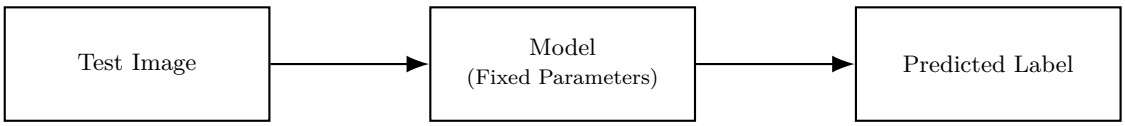
Definition 4. Each sample comes with additional annotations. e.g., category label

An assumption about the data that we are given is that the annotations (supervisory signal) are the insight that we seek to obtain from the data.
This is what we are more 'used to' when it comes to machine learning.

Learning Stages

This times the learning is divided into two stages:

1. **Training**: Use data with ground-truth labels to optimize model parameters (see playground.tensorflow.org)
2. **Testing**: Predict the ouput for a new data sample



Here we **don't** use our testing data to update the model parameters. We want to check wether or not our model **generalizes** to the new test data which might not be the case. If we go back to the example digit guessing demo; when we try to draw a number in a corner of the box, the model is not able to correctly predict. The reason for this is that the data that our model trained on had numbers only in the middle of the box → not trained to other cases.

Training set vs Test set

The training set and the test set should **always** be completely separate

Remark Using the observations (inputs) is occasionally possible and referred to as transductive learning (we will not do it in this course).

Furthermore, we also need that our training and test samples are drawn from the **same statistical distribution**

1.2 Linear Model

Let's us start with a simple supervised learning model, the goal is to start with this as our first building block and then add layer onto it (non-linearity, etc.).

Notation

Definition 5. We denote the ith **data sample** (input) in the collection of N samples as

$$x_i \in \mathbb{R}^d$$

a column vector of dimension D

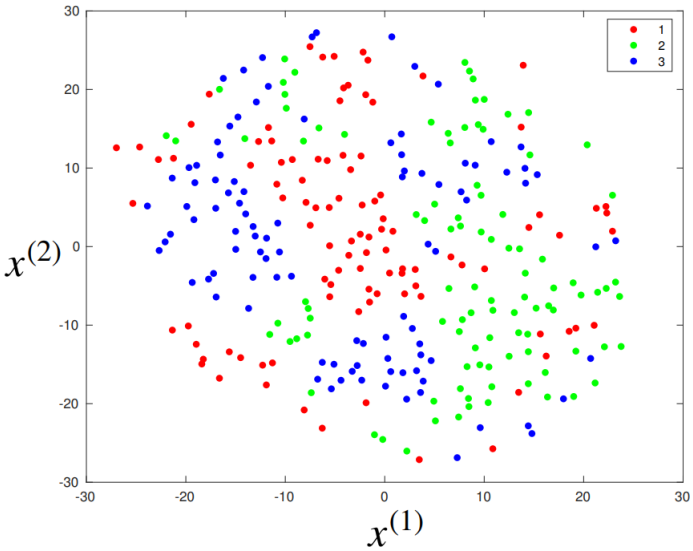
Definition 6. We denote the ith label (output) in the collection of N samples as

$$y_i$$

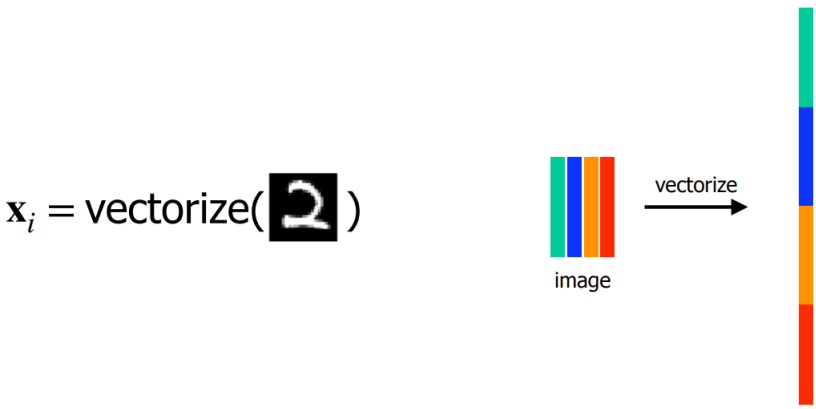
For **classification**, y_i represents the category the samples belongs to. Whereas for **regression**, y_i can be a single continuous value, or a column vector of dimension C.

Example

- In the following toy example, x_i is one 2D point, and thus $D = 2$
- y_i is a discrete value indicating the class (color), i.e., 1, 2, or 3



As another example, we can take the digit recognition example, x_i is a grayscale image. If it has height H and width W, then $D = H \cdot W$.



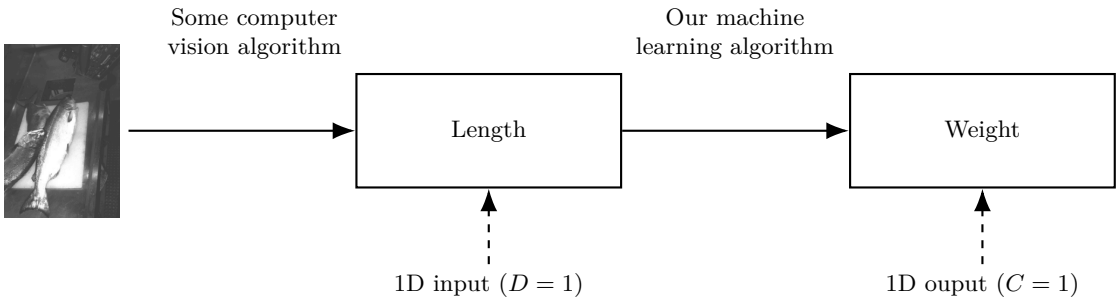
What we are essentially doing is transforming our image (which is a matrix in our case) to a vector by mapping each element of this message to a 'row' of the vector (this is the same a when we store a matrix in a row-major way).
In this case of digit recognition, the output y_i is a single discrete value indicating the digit, e.g. $y_i = 2$

Our first machine learning model

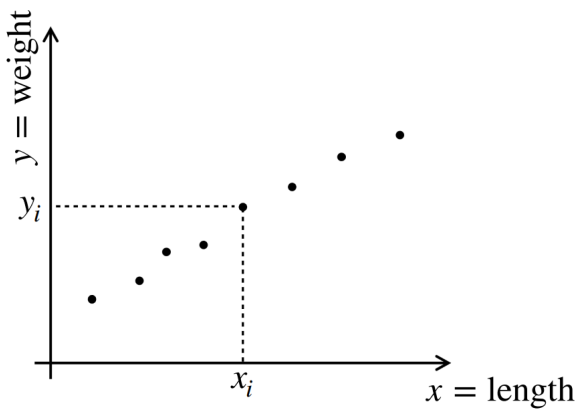
To start things off, let us create the **simplest** model possible:

- A single input dimension, $D = 1$
- A single output dimension, $C = 1$

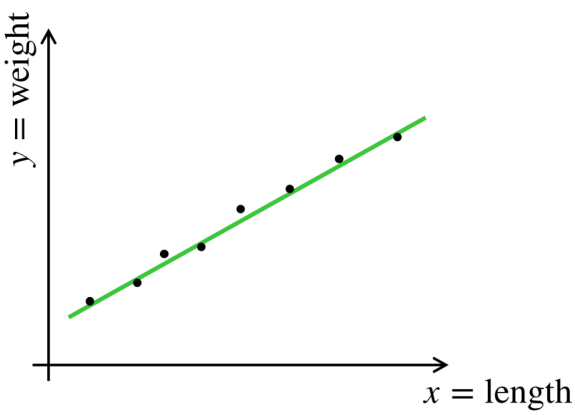
Our goal is to solve a regression task (the output is continuous). We seek to predict the weight of a fish given its length:



Given N annotated data samples $\{(x_i, y_i)\}$ with (length, weight) measurements, we can plot the data ($N = 8$ here):



We can see here that there is a kind of trend going on in this data set, we seek to approximate it with a line:



Once we have defined the line based on the N training samples, we can use it to predict the weight of any new length (again, assuming the test data follows the same distribution as the training data).

A simple parametric model: The line

Now let us think a bit more about the model, what is it made of? To be able to define a line we need 2 parameters:

- the y-intercept, we will call it $\omega^{(0)}$
- The slope $\omega^{(1)} = \frac{\Delta y}{\Delta x}$

With this two parameters, we can express any line

$$y = \omega^{(0)} + \omega^{(1)} x$$

This relation above **is** our model.

Line fitting

Now that we have our model, let us acutally fit our model to some data. Given N pairs $\{(x_i, y_i)\}$, the goal is to find the line that passes through these observations. But is this possible? Not really there is always noise in our data. The goal of fitting here is to find the line that **best fits** this observation and if you remember linear algebra, we actually have some tools to do so.

1D Linear Regression training

In essence, fitting a line consists of finding the best line parameters $\omega^{(0)*}$ and $\omega^{(1)*}$ for some given data. This corresponds to the training stage:

- Given N training pairs $\{(x_i, y_i)\}$, we aime to find $\left(\omega^{(0)}, \omega^{(1)}\right)$, such that the predictions of the model

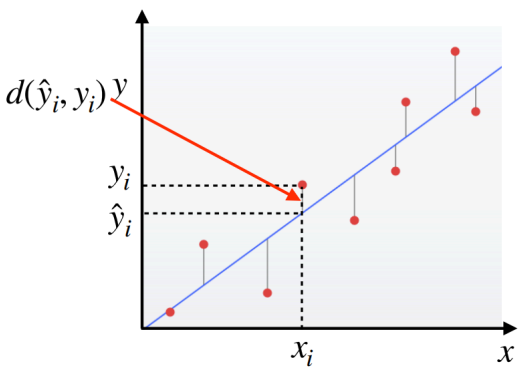
  $$\hat{y}_i = \omega^{(0)} + \omega^{(1)} x_i$$

  are close to the true values y_i

We then need to define a measure of closeness, or conversely an error, between y_i and $\hat{y}_i$.

As seen in linear algebra, a natural measure of the error is the Euclidean distance (also called L_2 norm)

$$d(\hat{y}_i, y_i) = \sqrt{(\hat{y}_i - y_i)^2}$$



In practice, one prefers to use the square of this distance:

$$d^2(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

With this definition, traning can be translated as a least-square problem minimization problem:

$$\left(\omega^{(0)*}, \omega^{(1)*}\right) = \text{argmin}_{\omega^{(0)}, \omega^{(1)}} \frac{1}{N} \sum_{i=1}^{N} d^2(\hat{y}_i, y_i)$$

Where $\hat{y}_i$ depends on $\omega^{(0)}$ and $\omega^{(1)}$ A nicer way which will be useful when using multi-dimension model, is to write our model as a matrix W such that

$$\hat{y}_i = W x_i$$

$$x_i \in \mathbb{R}^2$$

In our case as we have y being a one dimensional vector, we don't have to write our W as a matrix, a vector is sufficient:

$$x_i = \begin{pmatrix} 1 \\ x_i \end{pmatrix} \in \mathbb{R}^2 \qquad w = \begin{pmatrix} w^{(0)} \\ w^{(1)} \end{pmatrix} \in \mathbb{R}^2$$

Which gives us:

$$R(w) = \frac{1}{N} \sum_{i=1}^{N} \left(w^T x_i - y_i\right)^2 = \frac{1}{N} \sum_{i=1}^{N} \left(x_i^T w - y_i\right)^2$$

As said above, in general, an input observation x_i is not represneted by a single value. If we take a grayscale image, it can be represented by an $W \cdot H$ dimensional vector

$$x_i \in \mathbb{R}^{28 \cdot 28} = \mathbb{R}^{784}$$

Plane

For one dimension we had a line, for two dimension we then have... a **plane**!!

Remark 2. Remember to put the screen of the plane here

We can define our plane as:

$$y = w^{(0)} + w^{(1)} x^{(1)} + w^{(2)} x^{(2)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \end{pmatrix}$$

All the thing we said to a one dimension fitting is translated into two dimension: Given N noisy plan $\{(x_i, y_i)\}$, where $x_i \in \mathbb{R}^2$, find the plane that best fit these observations. Of course as you could have guess, this can be generalized to higher dimensions.

- In dimension D, we can write

$$y = w^{(0)} + w^{(1)} x^{(1)} + w^{(2)} x^{(2)} + \cdots + w^{(D)} x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

Ultimately, whatever the dimension, we can write

$$y = w^T x$$

With $x \in \mathbb{R}^{D+1}$, where the extra dimension contains a 1 to account for $w^{(0)}$.

Multi-input linear regression: Training

Because the output reamins to one dimension, we can use the same least-square loss function as before, and write training as:

$$\min_w \frac{1}{N}\sum_{i=1}^{N} d^2(\hat{y}_i, y_i) \iff \min_w \frac{1}{N}\sum_{i=1}^{N}\left(x_i^T w - y_i\right)^2$$

| *Note* This has the same form as in the one dimension input case

Solving this minimization problem involves computing its derivatives w.r.t. the model parameters (w) (by using the gradient descent).

1.3 Derivatives and Gradients

Definition 7. The derivative of a function $R(w)$ of a single variable w is the rate at which R changes as w changes

Remark 3. Be careful at this definition, if you have done cs-328 you'll know that derivative as in machine learning and many other topics, use auto-differentiation which is really "different" as how analysis defined derivative. (the 2023 exam asked to derive the load instruction...)

The derivative can also be measured for an infinitesimal change in w, startingfrom a point $\overline{w}$, we can write it as:

$$R'(\overline{w}) = \frac{dR}{dw} = \lim_{\Delta w \to 0} \frac{R(\overline{w} + \Delta w) - R(\overline{w})}{\Delta w}$$

Minimizing a function

In machine learning, we often seek to minimize a function (i.e., the loss function). To minimize such a function, we seek a point where the derivative vanishes, i.e., we search for w^* such that

$$\frac{dR(w^*)}{dw} = 0$$

For some simple, convex functions, w^* can be obtained algebraically by solving the corresponding equation

| *Remark* This is referred to as closed-form solution

Remark 4. As former analysis students, we see functions as nice curves that we can draw on a graph which can be specified using usual mathematical language (for instance $f(x) = \sin x$). In context of machine learning and auto-diff, this is not always true. Being able to algebraically solve an equation is something 'rare'.

For more complicated non-convex functions, there are typically multiple, local minima

Dealing with multivariate functions

This is nice for one dimension but even our simple case of a model with one dimension had already two parameters $(w^{(0)}, w^{(1)})$. In practice, we typically have more

than one parameter. To handle this, we rely on the function **gradient**. The gradient is a multi-variable generalization of the derivative.

- For a function with D parameters, the gradient is a D-dimensional vector
- Each element of this vector is the partial derivative of the function w.r.t. a single variable, i.e. the derivative of the function w.r.t. one variable while keeping the other constant

Definition 8. Gradient The gradient of a function $R(w)$ with $w \in \mathbb{R}^d$ is denoted by

$$\nabla_w R(w) = \begin{pmatrix} \frac{\partial R}{\partial w^{(1)}} \\ \frac{\partial R}{\partial w^{(2)}} \\ \vdots \\ \frac{\partial \mathbb{R}}{\partial w^{(D)}} \end{pmatrix}$$

Where $\frac{\partial R}{\partial w^{(j)}}$ denotes the partial derivative w.r.t. variable $w^{(j)}$.

Properties of gradient

- The gradient at a point w has the direction of greatest increase of the function w or in other word points to the **steepest** 'slope' of the function at the point v

You can see with this property why the gradient may be useful when looking for minima of a function. If the gradient give us the steepest slope of the function at a given point, when going down this slope, this means that we are going **down** the fastest way possible.

Little remark for the future

This is nice, going down the slope will make our loss function decrease. But what if it converges onlty to a local minima and not a global one? Doing a gradient descent at a local minima won't work or at least a 'vanilla' gradient descent. A solution to this would be gradient descent with momentum which i don't know if we will see later.

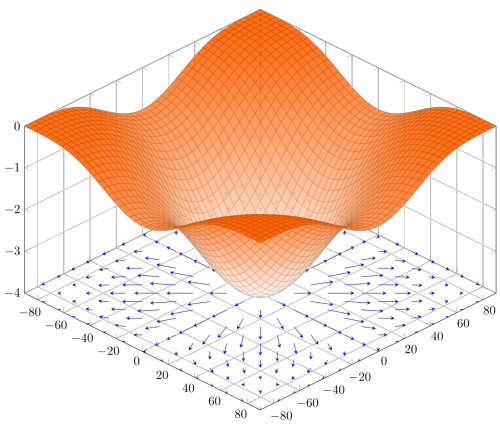


Figure 1.1: The gradient of the function $f(x, y) = (\cos^2 x + \cos^2 y)^2$ depicted as a projected vector field on the bottom plane. (wiki)

Linear regression: Solution

If we retake the least-square loss function we defined previously

$$R(w) = \frac{1}{N}\sum_{i=1}^{N}\left(w_i^T w - y_i\right)^2$$

This is a convex function of w, what this means is that the gradient is 0 only at the global minima. We therefore search a parameter w^* such that

$$\nabla_w R(w^*) = 0$$

Gradient Computation

Before solving our equation, let us first compute our equation $\nabla_w R(w)$. Remember first that $R(w)$ is an average over the N training samples. This means for us that we have N times the same equation for each i, which makes the gradient failry easy to compute, we only have to compute:

$$\nabla_w\left(x_i^T w - y_i\right)^2$$

By using the chain rule, the function $\left(x_i^T w - y_i\right)^2$ has the form $f(g(w))$, with $f(g) = g^2$. We can compute its gradient as $\frac{df(g)}{dg}\nabla_w g(w)$.

The last trick that we need is the following:

Theorem 1.
Given two vectors $a, b \in \mathbb{R}^D$

$$\nabla_b\left(a^T b\right) = a$$

Proof First, recall what the dot product is:

$$a^T b = \sum_{i=1}^{D} a_i b_i$$

This give us therefor by definition of the gradient:

$$\nabla_b\left(a^T b\right) = \begin{pmatrix} \frac{\partial a_1 b_1}{\partial b_1} \\ \frac{\partial a_2 b_2}{\partial b_2} \\ \vdots \\ \frac{\partial a_D b_D}{\partial b_D} \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_D \end{pmatrix} = a$$

Useful link Useful rules for matrix (or vector) computations, can be found in the Matrix cookbook:

http://www2.imm.dtu.dk/pubdb/edoc/imm3274.pdf

Using everything we said before, the gradient of the function $R(w) = \frac{1}{N}\sum_{i=1}^{N}\left(x_i^T w - y_i\right)^2$ can be computed as:

$$\begin{aligned}\nabla_w R &= \frac{1}{N}\sum_{i=1}^{N}\frac{\partial\left(x_i^T w - y_i\right)^2}{\partial\left(x_i^T w - y_i\right)}\cdot\frac{\partial\left(x_i^T w - y_i\right)}{\partial w}\\ &= \frac{2}{N}\sum_{i=1}^{N}\left(x_i^T w - y_i\right)\cdot x_i\end{aligned}$$

Solution

To obtain the solution, we search for a w^* such that

$$\nabla_w R(w^*) = \frac{2}{N}\sum_{i=1}^{N} x_i\left(x_i^T w^* - y_i\right) = 0$$

Note The x_i has moved to the front but this make no difference as $\left(x_i^T w - y_i\right)$ is a scalar

If we expand here what is inside the sum we have

$$x_i(x_i^T w^*) - x_i \cdot y_i$$

If we remove the constant factor $\frac{2}{N}$. Our solution has the following equation

$$\left(\sum_{i=1}^{N} x_i x_i^T\right) w^* = \sum_{i=1}^{N} x_i y_i$$

Okay, first let us look a bit more into the dimension of our equation. First $x_i \cdot x_i^T$ gives us a $(D+1)\times(D+1)$ matrix. w^* is a $(D+1)\times 1$ vector and $\sum_{i=1}^{N} x_i y_i$ is a $(D+1)\times 1$ vector.

$$\underbrace{\left(\sum_{i=1}^{N} x_i x_i^T\right)}_{(D+1)\times(D+1)}\underbrace{w^*}_{(D+1)\times 1} = \underbrace{\sum_{i=1}^{N} x_i y_i}_{(D+1)\times 1}$$

Matrix form

As said before, we prefer to use matrix when dealing with machine learning in general, you can also see here that we are doing a sum over product which kind of look like a matrix multiplication. Let us now group all $\{x_i\}$ and all $\{y_i\}$ in a matrix and a vector:

$$x = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_N^T \end{pmatrix} = \begin{pmatrix} 1 & x_1^{(1)} & x_1^{(2)} & \cdots & x_1^{(D)} \\ 1 & x_2^{(1)} & x_2^{(2)} & \cdots & x_2^{(D)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N^{(1)} & x_N^{(2)} & \cdots & x_N^{(D)} \end{pmatrix} \in \mathbb{R}^{N\times(D+1)}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^N$$

With this new notation, we can re-write the solution as:

$$X^T X w^* = X^T y$$

Sounds familiar to you? this is the definition of the least-square solution we have seen in linear algebra.

kind of proof?

A nice thing I want to put here which is not part of the course is how do we come up to the relation $X^T X w^* = X^T y$. What we are trying to do (from the beginning of this section), is to minimize the relation $\|Xw^* - b\|^2$. By definition of the scalar procut we have the following:

$$\begin{aligned}\|Xw^* - y\|^2 &=< Xw^* - y, Xw^* - y > \\ &=< Xw^*, Xw^* > - < X^*, y > - < y, Xw^* > + < y, y > \\ &= w^{*T} X^T X w^* - 2y^T X w^* + \|y\|^2\end{aligned}$$

As we want to minimize, we will take its derivative, which make the $\|y\|^2$ goes away:

$$\frac{\partial}{\partial x} w^{*T} X^T X w^* - 2y^T X w^* + \|y\|^2 = 2X^T X w^* - 2X^T y$$

As we are looking to minimize we seek it to be equal to 0 which gives us:

$$X^T X w^* = X^T y$$

The **solution** of this system is given as:

$$w^* = \left(X^T X\right)^{-1} X^T y = X^\dagger y$$

Where $X^\dagger$ is known as the Moore-Penrose pseudo-inverse of X (or directly the pseudo-inverse)

kind of proof II?

So if I understood correctly what the wiki says, the pseudo-inverse $A^\dagger$ and A^+ is essentially the same. The definition of the pseudo-inverse of a matrix A A^+ satisfy all of the four criterias, known as the Moore-Penrose condition:

$$\begin{aligned}AA^+A &= A \\ A^+AA^+ &= A^+ \\ \left(AA^+\right)^* &= AA^+ \\ \left(A^+A\right)^* &= A^+A\end{aligned}$$

I won't explain what the two lasts means (because I don't know), but why $X^\dagger = \left(X^T\right)^{-1} X^T$? Let us define B such that:

$$B = \left(X^T X\right)^{-1} X^T$$

If we multiply by X:

$$XB = X\left(X^T X\right)^{-1} X^T$$

And by X again but on the other side

$$XBX = X\left(X^T X\right)^{-1} x^T X$$

Which simplify to:

$$XBX = X$$

For the second criteria, retake our B and multiply by X:

$$\begin{aligned}BX &= \left(X^T X\right)^{-1} X^X X \\ &= I\end{aligned}$$

This directly implies that:

$$\underbrace{BX}_{I} B = B$$

And if we had proven the two last criteria we could have concluded that $B = \left(X^T X\right)^{-1} X^T = X^\dagger$.

Test time

As mentioned before, once we have the optimal parameters w^*, we can predict $\hat{y}_t$ for any new value x_t. This predicted value is given by:

$$\hat{y}_t = \begin{pmatrix} 1 \\ x_t \end{pmatrix}^T w^* = (w^*)^T \begin{pmatrix} 1 \\ x_t \end{pmatrix}$$

Evaluation metrics for regression

We can predict but is our prediction good? To be able to measure this we need two things:

- Test data
- An evaluation metrics

We have already spoke about the test data, seperate from the training data, doesn't change the parameters of the model... On the other hand, we have never seen any evaluation metrics before, a value that could tell us wether or not our model is performing well out there in the real world.

We have defined a loss function before, why not use it as the evaluation metrics? This function is 0 if the model matches perfectly with all data points, the greater the loss function the greater the offset to the correct value our model is predicting.

Mean Squared Error (MSE)

Definition 9. Mean Squared Error

$$\text{MSE} = \frac{1}{N_t} \sum_{i=1}^{N_t} (\hat{y}_i - y_i)^2$$

Where $\hat{y}_i$ is the prediction for the test sample i and y_i is the corresponding ground-truth value.

We can also use the Square-root of the MSE.

Mean Absolute Error (MAE)

Definition 10. Mean Absolute Error

$$\text{MAE} = \frac{1}{N_t} \sum_{i=i}^{N_t} \|\hat{y}_i - y_i\|$$

Maybe it can be more intuitive to look at the error as a pourcentage:

Definition 11. Mean Absolute Percentage Error

$$\text{MAPE} = \frac{1}{N_t}\sum_{i=1}^{N_t}\left\|\frac{\hat{y}_i - y_i}{y_i}\right\|$$

1.3.1 Interpreting a Linear Model

First an important thing to check we testing our model is comparin the error value of on the training data **and** for the test data. If those two are close this means that our model effectively can generalize to other data. If for instance our model as a very low error on the training data and a high error for the test data, usually this means that the model is **overfitting**.

The other intresting thing with a linear model is that each value of the model $w^{(i)}$ directly maps to an attribute of the input. If we take the example of the UCI Wine Quality dataset with the following data: After having trained our model we have as the final RMSE:

fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

Table 1.1: Wine quality dataset sample

- 0.65 for training data
- 0.63 for test data

Which is good for us, this means as said before that our model can generalize to other wine. Now let us see what is the parameters of our model:

	Coefficient
fixed acidity	0.017737
volatile acidity	-0.992560
citric acid	-0.139629
chlorides	-1.590943
free sulfur dioxide	0.005597
total sulfur dioxide	-0.003520
density	0.768590
pH	-0.4374144
sulphates	0.812888
alcohol	0.301484

Table 1.2: Wine quality parameters

Having a coefficient close to 0 means that this input is completly 'irrelevant' for our ouput (it is independant as in the probability sense).

Remark 5. Warning: the magnitude of a coefficient will depend on the magnitude of the corresponding feature/attribute:

- A coefficient might be very small imply to compensate for the fact that the range of the feature is very large
- This can be addressed by normalizing the data

1.3.2 Linear model with multiple outputs

Until now we have assumed that the ouput was a single value, i.e., $y_i \in \mathbb{R}$. But in practice, one may want to output multiple values for a given input, i.e., $y_i \in \mathbb{R}^C$ with $C > 1$ **Model**

To ouput multiple values, the linear model connot just relyu on a vector w, because the product $w^T x_i$ yields a single value. So let us a matrix $W \in \mathbb{R}^{(D+1)\times C}$ such that:

$$\hat{y}_i = W^T x_t = \begin{pmatrix} w_{(1)}^T \\ w_{(2)}^T \\ \vdots \\ w_{(C)}^T \end{pmatrix}$$

Since the ouputs are multi-dimensional, we need to slightly modify the loss function, the multi-dimensional counterpart of the 1D least-square loss is the squared Euclidean distance. Therefore, we can write multi-ouput linear regression as:

$$\min_W \sum_{i=1}^N ||W^T x_i - y_i||^2$$

Multi-ouput linear regression: Gradient

We have our loss function but computing its gradient seems a bit more sophisticated as how we did in the single-ouput linear regression. To compute its gradient, we make use of the following two properties:

- Norm and derivative:

$$||a||^2 = a^T a$$

  And thus:

$$\nabla_a ||a||^2 = 2a$$

- Matrix derivative:

$$\nabla_B \left(a^T B\right) = a$$

Thuse, using the chain rule (and replacing $\left(W^T x_i - y_i\right)$) with $\left(X_i^T W - y_i^T\right)$, we obtain the gradient:

$$\nabla_W R = 2 \sum_{i=1}^N x_i \left(x_i^T W - y_i^T\right)$$

Setting it to zero means that:

$$\underbrace{\left(\sum_{i=1}^N x_i x_i^T\right)}_{(D+1)\times(D+1)} \underbrace{W^*}_{(D+1)\times C} = \underbrace{\sum_{i=1}^N x_i y_i^T}_{(D+1)\times C}$$

For the 'training' part, we can again group in a matrix $X \in \mathbb{R}^{N\times(D+1)}$, but the ouputs now need to be grouped in a matrix (and not a vector) as

$$Y = \begin{pmatrix} y_1^T \\ y_2^T \\ \vdots \\ y_N^T \end{pmatrix} = \begin{pmatrix} y_1^{(1)} & y_1^{(2)} & \cdots & y_1^{(C)} \\ y_2^{(1)} & y_2^{(2)} & \cdots & y_2^{(C)} \\ \vdots & \vdots & \ddots & \vdots \\ y_N^{(1)} & y_N^{(2)} & \cdots & y_N^{(C)} \end{pmatrix}$$

Then, following the same strategy as before, we have:

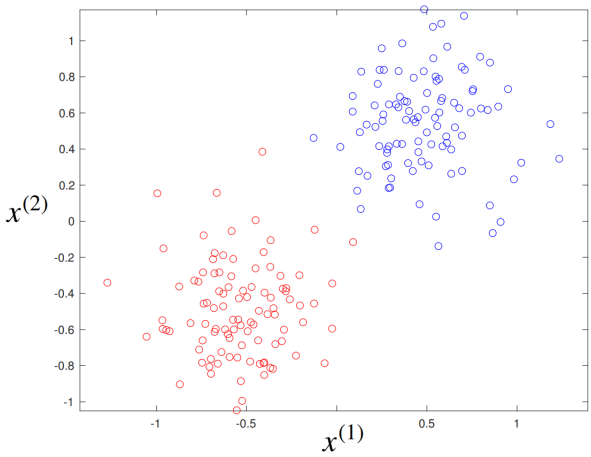
$$W^* = \left(X^T X\right)^{-1} X^T Y = X^\dagger Y$$

1.3.3 Linear Classification

The goal for now will the same as before however this time instead of having to output a continuous vector, the output will be a class (which is represented by an integer).

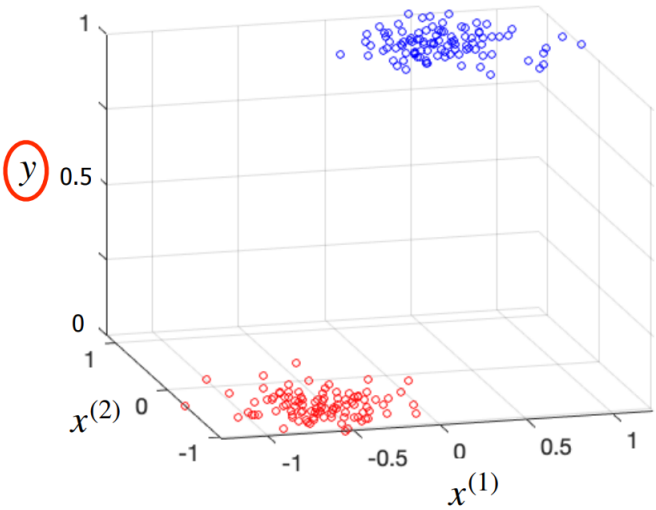
Binary Classification

The easiest classification model we can make is a binary classification one. The output of our model is either 0 or 1.
If we take the following data sets:



- We have 2 dimensions input
- The label $y \in \{-1, 1\}$, or $y \in \{0, 1\}$, is the 1D output
- The sample with label 1 are called **positive** samples
- The sample with label -1 (or 0) are called **negative** samples

But if I have this data in a 2D space, I can also map it to a 3D space by adding the y label into our space. The label acts as the output dimension, just as in a regression problem:



This means that we can now look at this problem as a regression task, one can tackle classification as a regression task!

Example (from Andrew Ng's course)

The goal is to classify a tumor as benign vs malignant, here's the data:

But we can still create a model on this data, a linear model which can predict the label as:

$$\hat{y} = w^{(0)} + w^{(1)}x$$

If $\hat{y}$ is small there is a high probability of being and vis versa.

To fit the linear model to the training data, the same strategy as for the linear regression can be used:

Given N pairs $\{(x_i, y_i)\}$, we can use the least-square loss to write:

$$\min_W \sum_{i=1}^{N} \left(x_i^T W - y_i\right)^2$$

Because it has exactly the same formulation as before, the solution is exactly the same, i.e.

$$w^* = \left(X^T X\right)^{-1} X^T y$$

Because we still need to ouput a **label**, we will transform our continuous value by using a threshold, for instance:

$$\text{label} = \begin{cases} 1 \text{ if } \hat{y}_t \geq 0.5 \\ 0 \text{ otherwise} \end{cases}$$

Where $\hat{y}_t = x_t^T w^*$.

Remark A general rule when using least-square binary classification is that: the boundary between our two class is of dimension D. when we had data of dimension 1 the decision boundary was a point (i.e. 0.5 above). If our data are in two dimensions, the boundary will be a line.

1.4 Linear Model for Classification

As said in the previous lecture, a simple linear model, represents an hyperplane. In dimension D, we can write:

$$y = w^{(0)} + w^{(1)}x^{(1)} + \cdots + w^{(D)}x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

Ultimately, whatever the input dimension, we can write:

$$y = w^T x$$

with $x \in \mathbb{R}^{D+1}$, where the extra dimension contains a 1 to account for $w^{(0)}$.

1.4.1 Loss Function ad Empricial Risk

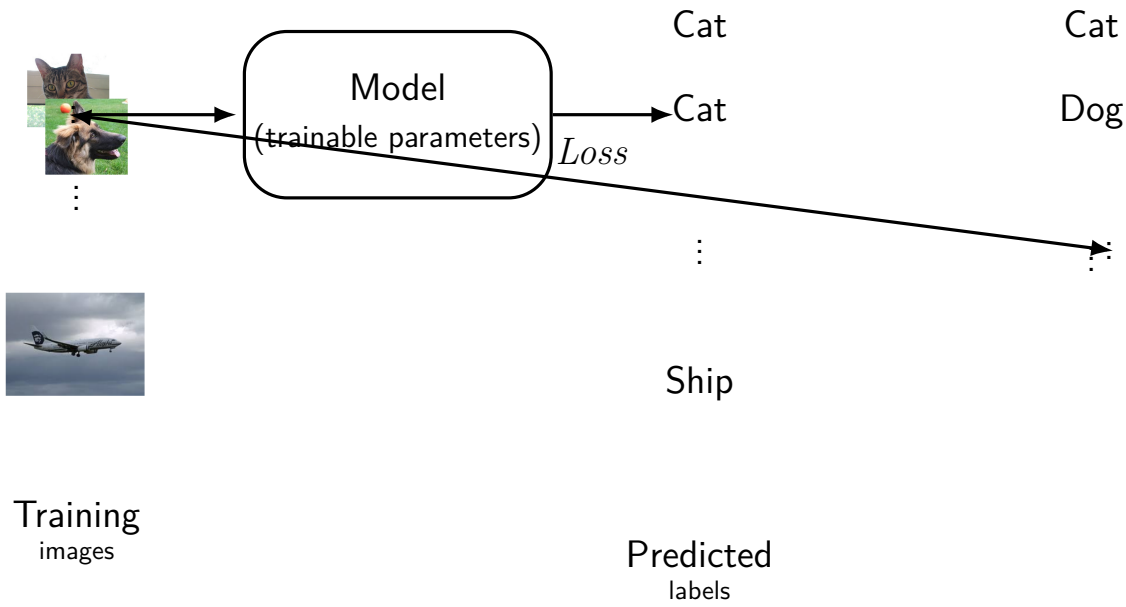
Remark 6. Those are general machine learning concept

Loss function

Definition 12. The loss function $l(\hat{y}_i, y_i)$ computes an error value between the prediction and the true value

This is a general machine learning conecpt, this is not used only for linear regression, we will see different loss functions in the upcoming lectures.

The goal of the loss function is to **measure** how **wrong** our prediction is. Meaning that if we were to train a model in order to guess wether it is a dog, cat, ship, plane, etc.. The loss function will tell us how wrong are we. (notice that even with classification model we will need to have a loss function (even is it will measure a 'yes' or 'no' type question)).



Loss function: Examples

So far, we have already seen quite a lot of loss functions for linear regression:

- Single ouput dimension: $l(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$
- Multiple output dimension: $l(\hat{y}_i, y_i) = ||\hat{y}_i - y_i||^2$

In principle, other loss function could be used for regression, for example, using the MAE evaluation metric

$$l(\hat{y}_i, y_i) = \|\hat{y}_i - y_i\|$$

As we will see later today and in future lectures, other loss function can be used for classification.

Empirical risk

Definition 13. Empirical Risk
Given N training sample $\{(x_i, y_i)\}$, the empirical risk is defined as:

$$R(\{x_i\}, \{y_i\}, w) = \frac{1}{N} \sum_{i=1}^{N} l(\hat{y}_i, y_i)$$

Where $\{x_i\}$, and $\{y_i\}$ are the sets of training inputs and labels, respectively.

When we are training our model, the goal is to minimize the empirical risk – find the parameters w (e.g., $w^{(0)}, w^{(1)}$) that minimize the empirical risk.

Remark Note that the risk depends on w via $\hat{y}_i$.

Minimzing the risk

This is expressed as the optimization problem:

$$\min_w \frac{1}{N} \sum_{i=1}^{N} l(\hat{y}_i, y_i)$$

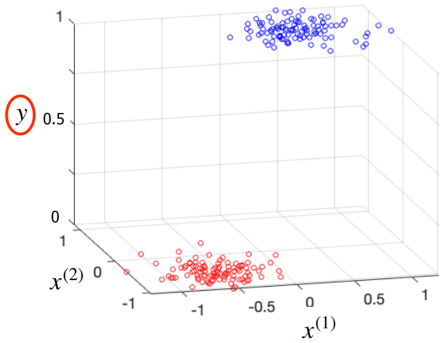
We can also write that the best parameters as a solution to the following equation:

$$w^* = \text{argmin}_w \frac{1}{N} \sum_{i=1}^{N} l(\hat{y}_i, y_i)$$

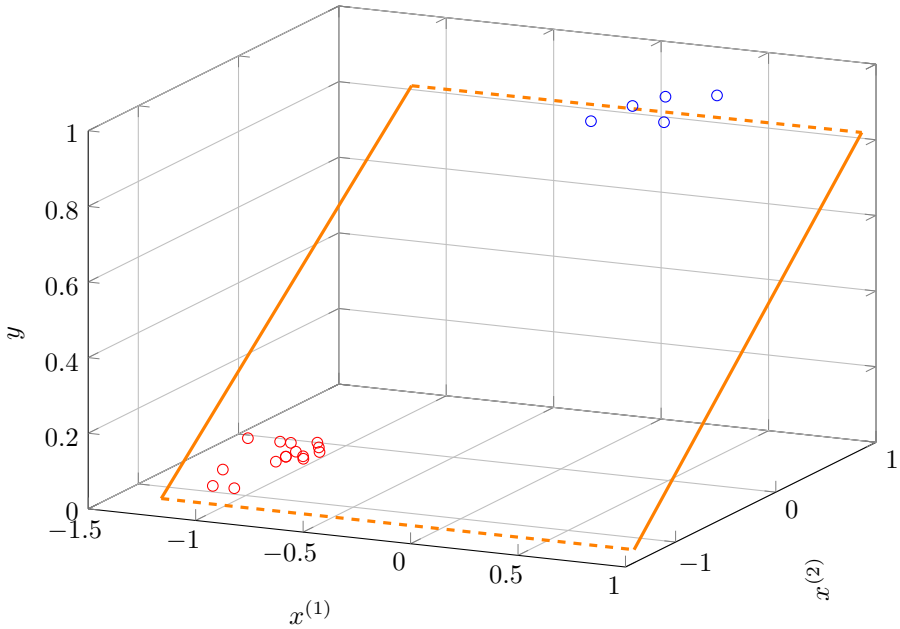
This is what we have done when training our linear regressor. Our least-square minimization problem is one example of empirical risk minimization, using the square loss.

From regression to classification

As said before, how we created classification model was directly based from a regression task. We took our training data and label, and mapped them into A $D + 1$ dimension. For instance given data point x with $x^{(1)}$ and $x^{(2)}$ and two calsses 0 and 1, we would ouput our data as:



Then we would train our regression model with our data which will give us a plane:



The linear model encodes the plan that passes through the samples (orange plane). To turn this into a classification model, we need to find the **decision boundary**

Remark 7. I just want to emphasize what is going on here. We first try to model the data using a linear model of the form

$$f(x) = w^T x + b.$$

The parameters of this model are chosen to minimize a certain **loss function**, which measures how well the model fits the training data.

After this, the goal is to create a classification model from the model we just built. In binary classification, the decision boundary is defined by

$$w^T x + b = 0.$$

In general, for data in D dimensions, this equation defines a $(D - 1)$-dimensional hyperplane. In particular, when $D = 2$, the decision boundary is a straight line.

Adding non-linearity

The output of the linear model $w^T x$ is a continuous value. However classification model output a label (a discrete value). We therefore need to '*cast*' our value before output:

$$y = \begin{cases} 1 \text{ if } w^T x \geq 0.5 \\ 0 \text{ otherwise} \end{cases}$$

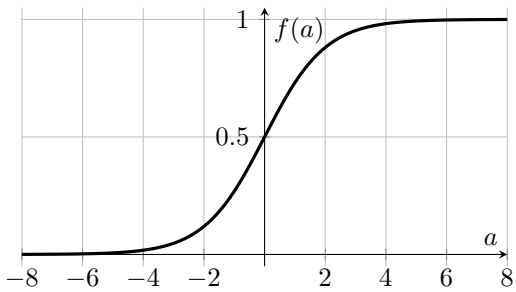
Step function: Drawback

However the gradient of the step function is not continuous, this means that using gradient descent won't work very well, optimizing become pretty hard.

> *Remark* The perceptron algorithm can nonetheless handle the two-class case, but we will not cover it in this course.

The better way is to use a '*smoother*' step function: the logistic sigmoid function:

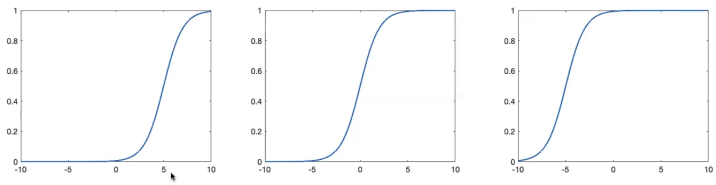
$$f(a) = \frac{1}{1 + \exp(-a)}$$



With a 1 D input x, applying the logistic function to the output of a linear model gives a prediction:

$$\hat{y} = \frac{1}{1 + \exp\left(-w^{(0)} - w^{(1)} x\right)}$$

Now an interesting thing to see is the effect of each parameters on our $\hat{y}$:



$w^{(0)} = -5$, $w^{(1)} = 1$ $w^{(0)} = 0$, $w^{(1)} = 1$ $w^{(0)} = 5$, $w^{(1)} = 1$

Probabilistic interpretation

In the binary case, with D dimensional inputs, we can write the prediction of the model as:

$$\hat{y} = \sigma\left(w^T x\right) = \frac{1}{1 + \exp(-w^T x)}$$

Such a prediction can be interpreted as the probability that x belongs to the positive class (or as a score for the positive class). The probability to belong the negative class is then computedd as $1 - \hat{y}$.

Logistic regression

Definition 14. The model whose prediction is defined as

$$\hat{y} = \sigma\big(w^T x\big)x = \frac{1}{1 + \exp(-w^T x)}$$

Is call a **logistic regression**

Remark Note that this name is slightly misleading, as this is truly a classification model, not a regression one

At its heart, logistic regression still use a linear model, but followed by a non-linearity to better encore the underlying binary classification problem.

Logistic regression: Training

Given N training samples, to learn the parameters w of the logistic regression model, we need to define a loss function. Instead of using the L_2 loss as in the least-square case, this is achieved by making use of the probabilistic interpreation.

Remark
- Because knowledge of probabilities is not required for this course, the derivation of the loss function that follows will be a bit hand-wavy
- Nevertheless, this loss remains intuitive: The loss is such that it aimes to find the parameters that maximize the probability of the correct class for each training sample

Specifically, the logistic regression loss is derived by first writing the following likelihood estimator:

$$p(y|w) = \prod_{i=1}^{N} \hat{y}_i^{y_i} \cdot (1 - \hat{y}_i)^{1-y_i}$$

Because, a product of terms between 0 and 1 can be unstable to optimize, we take the logarithm of this likelihood and take its negative to define an empricial risk to minimize, written as

$$R(w) = -\sum_{i=1}^{N} (y_i \ln(\hat{y}_i) + (1 - y_i)\ln(1 - \hat{y}_i))$$

This loss function is known as the cross-entropy (AICC II mentionned?)
Since y_i is either 0 or 1, the cross-entropy can be re-written as:

$$R(w) = \sum_{i \in \text{positive samples}} \ln(\hat{y}_i) - \sum_{i \in \text{negative sample}} \ln(1 - \hat{y}_i)$$

A very nice thing here for us is that, the **cross-entropy is a convex function**. But in this case, its minimization as no closed-form solution. This means that we will need to use an approximation: **gradient descent**!!

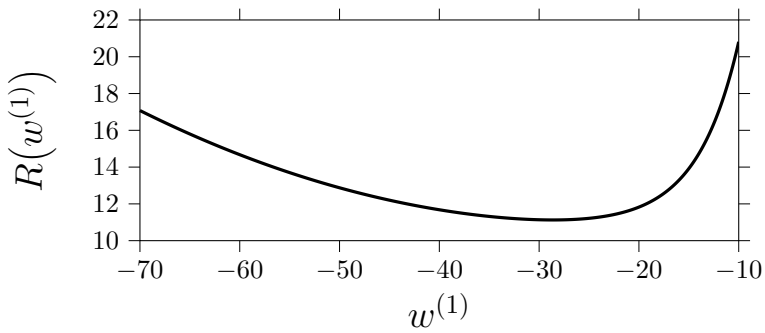


Figure 1.2: Cross entropy

Probabilistic Derivation

As we have now all did probability and statistics, we can still build it ourself. First, for us to train our model via the maximum likelihood estimator, we need to understand from where it comes from. We will do it as follow:

- Choosing a probabilistic model for $y|x$
- Writing the likelihood of the observed labels
- Maximizing it (equivalently minimizing the negative log-likelihood)

First, in logistic regression we have $\theta = (w, b)$. Where $w \in \mathbb{R}^D$ are the weights and b is the bias. (we can also see $\theta \in \mathbb{R}^{D+1}$).

Secondly, logistic regression assumes a probability model for the labels. For each data point x we have that:

$$P(Y = 1|x, \theta) = \sigma\big(\theta^T x\big)$$

What this says to us: given x, the label y folows a **bernoulli distribution**:

$$Y|x \sim \text{bernoulli}\big(\sigma\big(\theta^T x\big)\big)$$

Then given that we can compute the likelihood estimator as we used to. First defined it:

$$\begin{aligned}\mathcal{L}(\theta) &= P(y_1, y_2, \ldots, y_n|x_1, x_2, \ldots, x_n, \theta) \\ &= \prod_{i=1}^{n} P(y_i|x_i, \theta)\end{aligned}$$

If we develop this we get:

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} \sigma\big(\theta^T x_i\big)^{y_i} \big(1 - \sigma\big(\theta^T x_i\big)\big)^{1-y_i}$$

Now that we have defined it we seek to find the θ^* such that:

$$\theta^* = \arg\max_{\theta} \mathcal{L}(\theta)$$

And as we did in statistics, we can take the log of this function and computing the maximum on the log which is '*nicer*'. We therefore try to maximize:

$$\sum_{i=1}^{n} \big(y_i \log \sigma\big(\theta^T x_i\big) + (1 - y_i) \log\big(1 - \sigma\big(\theta^T x_i\big)\big)\big)$$

Intuition

- The data already exists
- We assume a **probabilistic model**
- The model has parameter θ
- We ask:

  For which θ would this observed data be the most likely

1.4.2 Iterative, Gradient-based optimization

The principle of gradient-based optimization and more generally iterative optimization is that: we have found a way to have a vetter solution based on a previous one. This means that if we iter on this 'way' a lot of time we will have like better(better(better $(\cdots(\text{base}))))$ which will give us not necessarly the best solution but a good enough solution.

Minimizing a function

With 1D function for which the solution cannot be computed in closed form, one can nonetheless use the derivative of this function to minimize it.

- Recall that it represents the slope at a given point, and thus indicates in which direction to move to reach a lower function value

Therefore the algorithm can be define pretty easily:

1. Initialize w_0 (randomly)
2. While not converged do
   (a) Update $w_k \leftarrow w_{k-1} - \nu \frac{dT(w_{k-1})}{dw}$

The ν defines the step size of each iteration. (often called the learning rate)

Convergence criteria

This is nice but we still need potential convergence criteria to be sure that our algorithm works effectively:

- Change in function values less than a given threshold: $\|R(w_{k-1} - R(w_k))\| < \delta_R$
- Change in parameter value less than a given threshold: $\|w_{k-1} - w_k\| < \delta_w$
- Maximum number of iterations reached, Meaning that we have no guarantee to have reached the minimum

Remark 8. In theory, one would need the function to has lipschitz continuous gradient in order to have stable steps. What this means is we will assume:

$$||\nabla R(x) - \nabla R(y)|| \leq L||x - y||$$

The rule mentionned above are more practical criteria. In practie we don't want to compute exact optimality, we so care more about:

- change in objective
- change in parameters

Our current criteria work well **because** the function is convex + smooth

When minimizing we have to be careful about:

Influence of ν Too large steps can lead our algorithm to start jumping around the 'solution' and oscillates till infinity. We are not able to have a magic formula to have a good η, each η is problem-dependent.

Influence of the initial w_0 Assuming that our function is convex, the algorithm will always converges to the same point. Choosing a good w_0 will still converge to the same point, but **faster**.

Minimizing a non-convex function

As in convex function, we can still use the same iterative algorithm as before. However this time. The output of our algorithm depends now on w_0. We can be stuck at local minimum and/or at saddle point.

Remark Note that this is problem-dependent and the values used here are very different from those used in the convex sample before.

In practice, we have more than one parameter, i.e. $w \in \mathbb{R}^{D+1}$, and we thus use the function gradient:

$$\nabla_w R(w) = \begin{pmatrix} \frac{\partial R}{\partial w^{(0)}} \\ \frac{\partial R}{\partial w^{(1)}} \\ \frac{\partial R}{\partial w^{(2)}} \\ \vdots \\ \frac{\partial R}{\partial w^{(D)}} \end{pmatrix}$$

Where $\frac{\partial R}{\partial w^{(j)}}$ denotes the partial derivative w.r.t. variable $w^{(j)}$

Recap: Properties of gradient

- The gradient at a point w has the direction of greatest increase of the function at w
- Its magnitude is the rate of increase in that direction

Gradient descent on multivariate function

The algorithm stays the same but we will use the gradient instead:

1. Initialize w_0 (e.g., randomly)
2. While not converged:
   (a) Update $w_k \leftarrow w_{k-1} - \eta \nabla_W R(w_{k-1})$

The learning rate η has the same effect as in the 1D case.

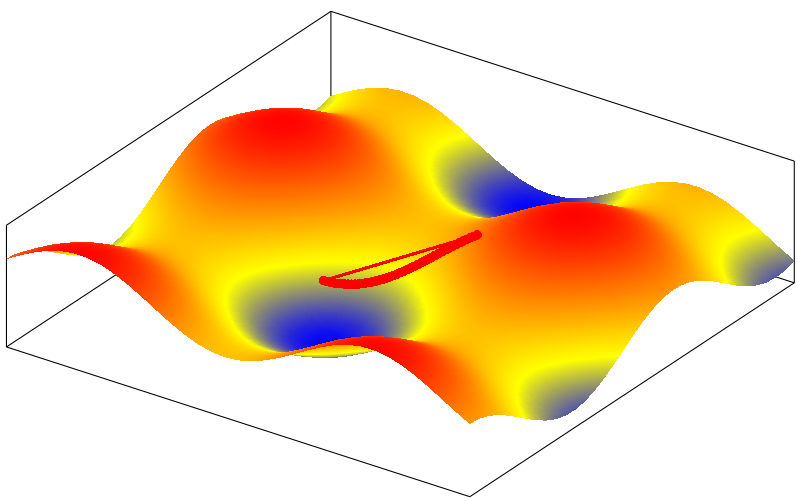
Convergence can again be measured by:

- Thresholding the change in function value
- Thresholding the change in paramters, e.g. $||w_{j-1} - w_k|| < \delta_w$

Minmizing a non-convex function Example: sin, cos based function of 2 variables

Let our function be:

$$R(w) = \sin\left(w^{(0)}\right) + \cos\left(w^{(1)}\right)$$



As in the 1D cases the same issues can arise:

- Different learning reate can yield different results
- Different Initializations can yield different results

Back to Logistic Regression

With binary ground-truth label y_i, the cross entropy can be written as:

$$\begin{aligned} R(w) &= \sum_{i \in \text{positive samples}} \ln(\hat{y}_i) - \sum_{i \in \text{negative sample}} \ln(1 - \hat{y}_i) \\ &= \sum_{i \in \text{positive samples}} \ln \sigma\big(w^T x_i\big) - \sum_{i \in \text{negative sample}} \ln\big(1 - \sigma\big(w^T x_i\big)\big) \end{aligned}$$

To minimize it via gradient descent, we need to compute its gradient. Using the chain rule, and after some derivation, we have:

$$\nabla_w \left(- \sum_{i \in \text{positive}} \ln \sigma\big(w^T x_i\big) \right) = \sum_{i \in \text{positive}} (\hat{y}_i - 1) x_i = \sum_{i \in \text{positive}} (\hat{y}_i - y_i) x_i$$

Similarly, for the negative examples, we have:

$$\nabla_w \left(- \sum_{i \in \text{negative}} \ln\big(1 - \sigma\big(w^T x\big)\big) \right) = \sum_{i \in \text{negative}} \hat{y}_i x_i = \sum_{i \in \text{negative}} (\hat{y}_i - y_i) x_i$$

Finally,this can be put back together to obtain:

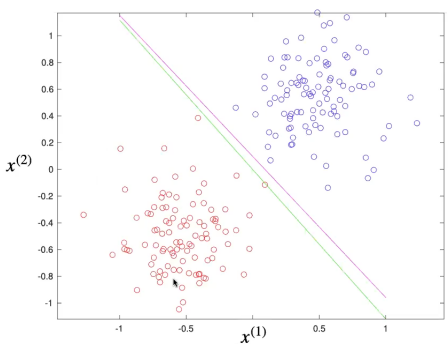
$$\nabla_w R(w) = \sum_{i=1}^{N} (\hat{y}_i - y_i) x_i$$

Where y_i is the true label for sample i, i.e. 1 for positive sample and 0 for negative samples

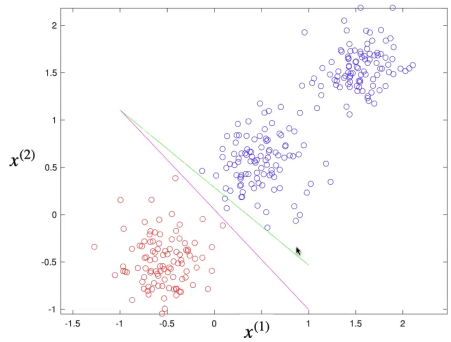
Intuitively, the gradient is given by the error of the current prediction $(\hat{y}_i - y_i)$, multiplied by the input.

Example

If we take the previous example and compare the least-square classification (in green) and the logistic regression in magenta, we get:



But now, with additional samples, the boundary remains correct:



Remark 9. As with linear regression, the learned coefficients of logistic regression can also be analyzed, see a discussion of this at `https://www.displayr.com/how-to-interpret-logistic-regression-coefficients/` (this is not part of the course)

Linear models

The real difference between these models is the loss function.

- The choise of loss function has an impact on the resulting algorithm
- Let's have a look at the intuition behind SVM

Margin

To have a better intuition of the SVM we first need to introduce the notion of **margin**:

Definition 15. The distance between the decition boundary and the nearest sample is called **margin**

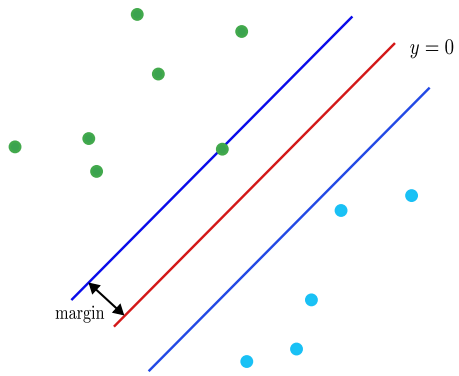


Figure 1.3: margin

Intuitively, a good decision boundary should maximize the margin. Such a boundary is determined by a subset of the data points, named support vectors (indicated with circles) when we have two classes what this essentially means is that our boundary will be centered between both our classes.

Support Vector Machine

The SVM classifier is a binary classifier based on the linear model as before:

$$\hat{y} = w^T x$$

It aims to maximize the margin while enforcing a correct classification for each training samples. This is formulated as a convex optimization problem that can be solved to optimality with standard iterative solvers

In the presence of additional samples, the decision boundary is unchanged. The goal of this model is not to have a '*weighted mean*' it's more finding the **best boundary** such that the score is minimize. The only vector that truly matter are the one close to the decision boundary.

Loss function

Ultimately, training an SVM can be written as:

$$\min_{w^{(0)},\widetilde{w}} \frac{1}{2C}\|\widetilde{w}\|^2 + \sum_{i=1}^{N} \max\Big(0, 1 - y_i\Big(w^{(0)} + \widetilde{w}^T x_i\Big)\Big)$$

- The term $||\quad \widetilde{w} \quad||^2$ corresponds to the inverse of the margin (distance between the decision boundary and the nearest sample); minimizing it seeks to maximize the margin
- The term:

$$\max\Big(0, 1 - y_i\Big(w^{(0)} + \widetilde{w}^T x_i\Big)\Big)$$

  is referred to as the **hinge loss** and encourages correctly classifying each sample.

1.5 Classification Metric

The goal now is to understand what is our model **behavior**. If it guesses correctly, if he is wrong, what type of wrong, etc...

Classification evaluation

Definition 16. We can first define the **confusion matrix** in the binary case (e.g., spam vs non-spam email):

	Pos	Neg
Yes	TP	FP
No	FN	TN
	P = TP + FN	N = FP + TN

Where:

- TP: True positive (number of samples correctly classified as positive)
- FP: False positive (number of samples incorrectly classified as positive)
- TN: True negative (number of sample correctly classified as negative)
- FN: False negative (number of sample incorrectly classified as negative)

Using this confusion matrix, we can study further the result of our model.

Example

Let us use a numerical example for a dataset with 500 positive and 500 negative samples:

	Pos	Neg
Yes	200	100
No	300	400
	P = 500	N = 500

Given this matrix, what we can first compute is the **accuracy**:

$$\text{Acc} = \frac{TP + TN}{P + N}$$

In our example we have:

$$\text{Acc} = \frac{200 + 400}{500 + 500} = 0.6$$

But is it enough? Not really let two classifiers:

	Pos	Neg
Yes	200	100
No	300	400
	P = 500	N = 500

	Pos	Neg
Yes	400	300
No	100	200
	P = 500	N = 500

The two classifiers (matrices) above yield the same accuracy but their behavior is very different:

- Left: Weak positive recognition rate, but strong negative recognition rate
- Right: Strong positive recognition rate, but weak negative recognition rate

Another classification metrics we can compute is called **Precision**:

Definition 17. Precision
The percentage of samples classified as positives that are truly positives:

$$\text{Precision} = \frac{TP}{TP + FP}$$

In our previous example, this would give us:

$$\text{Precision} = \frac{200}{200 + 100} = 0.667$$

(ekip)

Another metric is called **Recall**:

Definition 18. Recall
the percentage of positive samples that are correctly classified as positives:

$$\text{Recall} = \frac{TP}{P}$$

In our example:

$$\text{Recall} = \frac{200}{200 + 300} = 0.4$$

But still even with those metrics, we can still have different behavior and same metrics: let us take the two followings:

	Pos	Neg
Yes	200	100
No	300	400
	P = 500	N = 500

	Pos	Neg
Yes	200	100
No	300	0
	P = 500	N = 100

In this case, the two classifiers (matrices) above yield the same precision and recall (0.667 and 0.4) (the datasets are different). But in this case their behavior is very different. Accuracy in this case would be able to show the different behavior. The goal of this example is to show that the more metrics you have the better your intepretation of the model you will get

Other metrics

Definition 19. False positive rate
Percentage of negative samples that are incorrectly classified as positive:

$$\text{FP rate} = \frac{FP}{N}$$

Definition 20. F1 score
Single number that combines precision and recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

ROC curve

Many classifiers output a score/confidence for their prediction (e.g., $\hat{y}$ is a continuous value between 0 and 1 in logistic regression)

The decision between positive and negative can be computed by thresholding this score:

$$\text{label}_i = 1 \text{ if } \hat{y}_i \geq \tau$$

Where τ is a given threshold.

The Receiver Operating Characteristic (ROC) curve plots the true positive rate (recall) as a function of the false positive rate, obtained by varying the score threshold

Remark 10. Remember to put graph here as an example (slide 26)

The Area Under the ROC Curve (AUC) can act as a metric:

- The maximum AUC is 1
- Random predictions give an AUC of 0.5

1.5.1 From binary to multi-class classification

So far, three methods we have studied can only handle two classes: positive vs negative.

In general, one would typically like to disciminate between more than two categories. E.g., for image recognition. Let us now see how our three linear methods defined earlier can handle this scenario.

least-square classification

In essence, least-square classification treats the problem as a regression task

- The model predicts a continuous value $\hat{y} = w^{(0)} + w^{(1)}x$
- There is nothing that explicitly makes this value stop at 1, so we could continue the line further

The simplest way to go from binary case to handling multiple classes would then be to use one integer value for each category. (we then round the prediction of the model to the nearest integer to obtain the predicted class value).

This, however makes no sense, there is **no natural** order between the different categories (the prof trolled us). If we were to do so, this would really just comes down to a litteral regression task. Furthermore this would implies that guessing 4 instead of 3 would be better than guessin -1 and that's complete nonsense when speaking about classification tasks.

Dealing with multiple classes

The most common approach to modeling a multi-class problem consists of encoding the class label as a vector with a single 1 at the index corresponding to the category and 0's elsewhere

> *Intuition* We can see it as each label is a dimension in $\mathbb{R}^D$. A good classifier would then output a vector (as a guess) and this vector needs to have the same direction (or at least, near) the label.

This is referred to as one-hot encoding (or 1-of-C encoding)

Multi-ouput linear model

Predicting such a one-hot encoding is then similar to making the model output multiple values, as we saw for multi-ouput regression.

We therefore again use a matrix $W \in R^{(D+1)\times C}$ such that:

$$\hat{y}_i = W^T x_i = \begin{pmatrix} w_{(1)}^T \\ w_{(2)}^T \\ \vdots \\ w_{(C)}^T \end{pmatrix} x_i$$

multi-class classification is then similar to a multi-output regression problem:

We can then write training as:

$$\min_W \sum_{i=1}^{N} ||W^T x_i - y_i||^2$$

where each y_i is now a C-dimensional vector with a single one at the index corresponding to the class label, and zeros everywhere else.

Remark 11.
You can also see it as C independant binary classifier. You can seperate each $w_{(i)}^T$ and train them independetly and then joined them into W and you would get the same result. (In the case of the square loss).

The solution has the same form as in the regression case (obtained by zeroing out the gradient w.r.t. W), i.e.,

$$W^* = \left(X^T X\right)^{-1} X^T Y = X^\dagger Y$$

Given the optimal paramters, the prediction for a new sample is computed as:

$$\hat{y} = (W^*)^T x$$

The sample is then assigned to class k is:

$$\hat{y}^{(k)} > \hat{y}^{(j)}, \forall j \neq k$$

Or in other words the final label k is obtained as:

$$k = \arg\max_j \hat{y}^{(j)}$$

1.5.2 Decision Boundary

Remark 12. This following section applies to all linear classification models, not just least-square classification.

Decision boundaries

Definition 21. The decision boundary between two classes k and j is defined as the set of points x such that $\hat{y}^{(k)}(x) = \hat{y}^{(j)}(x)$

This corresponds to the $(D-1)$ dimensional hyperplane

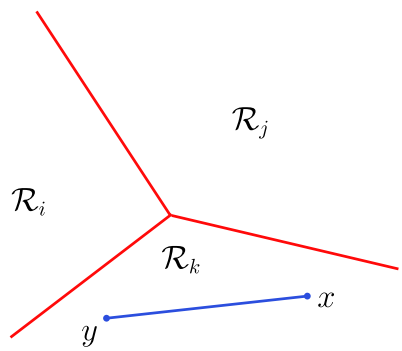


Figure 1.4: Decision boundary

The binary case is a special case of this definition.

A nice feature of those boundaries is that they are **convex**. Recall that a set S is said to be **convex** if for any pair of point $x, y \in S$ and any $\lambda \in [0, 1]$ we have that

$$\lambda x + (1 - \lambda)y \in S$$

Which means that the segment between x and y is a subset of S.

Decision Boundaries for $C > 3$ classes

For 1 and 3 this is quite intuitive but what happens when we have 4 classes, if we take the following data set: (put an image here) We would get this as decision boundary.

Example of least-square classification

Remember to put both the failure and the good case

Multi-class least-square classification: Drawbacks

As in the binary case, multi-class least-square classification suffers from the fact that the loss function does not reflect the underlying classification task.

In the binary case, we addressed this by adding a nonlinearity to the output of the linear model via the logistic function, leading to **logistic regression**. Maybe there is a better way to extends this to the multi-case scenario???

1.5.3 Multi-class logistic regression

As in the least-square classification case, we can use one-hot encodings to represent multi-class labels

Then, we again need to use a matrix $W \in \mathbb{R}^{(D+1)\times C}$ to represent the model parameters.

In this case, the probability for a class k is given by the softmax function:

$$\hat{y}^{(k)}(x) = \frac{\exp\left(w_{(k)}^T x\right)}{\sum_{j=1}^{C} \exp\left(w_{(j)}^T x\right)}$$

Multi-class logistic regression

The emprical risk can then be derived from the multi-class version of the cross entropy, which yield:

$$R(W) = -\sum_{i=1}^{N} \sum_{k=1}^{C} \hat{y}_i^{(k)} \ln \hat{y}_i^{(k)}$$

As in the binary case, a single $y_i^{(k)}$ is 1 for every sample i, so we really have one term per training sample. And also as in the binary case, training is done by minimizing this emprical risk using gradient descent.

Gradient for multi-class logistic regression

By following the chain rule and using the gradient of the softmax function, we can derive the gradient of the multi-class cross-entropy as (details in Bishop Chap.

4.3.4):

$$\nabla_W R(W) = \sum_{i=1}^{N} x_i(\hat{y}_i - y_i)^T$$

We have the same intuition as in the binary case, we multiply the error by the input.

Prediction

In order to predict, given a new input sample x, the probability/score for any class k is computed via the softmax function as:

$$\hat{y}_i^{(k)}(x) = \frac{\exp\left(\left(w_{(k)}^*\right)^T x\right)}{\sum_{j=1}^{C} \exp\left(\left(w_{(j)}^*\right)^T x\right)}$$

The final class label is then predicted as:

$$k = \arg\max_j \hat{y}^{(j)}(x)$$

Remember to put screenshot slide 53

Evaluation metrics for multi-class classification

As in the binary case, we can compute a confusion matrix which is pretty intuitive, for instance here with 3 classes (UCI Iris dataset)

	Iris-setosa	Iris-versicolor	Iris-virginica
Iris-setosa	12	1	1
Iris-versicolor	0	16	0
Iris-virginica	0	1	16

Where the row are the Actual and the collumn are the predicted value. This means that we have only once predicted Iris-versicolor when in fact it was a Iris-setosa

Metrics for multi-class problems

We still have the **Accuracy** and the **recall**, **FP-rate** and **F1-score**

Accuracy

A global accuracy can be obtained by summing the correct predictions for all classes (diagonal values of the confusion matrix), and dividing this by the total number of samples (sum of all entries in the confusion matrix)

Precision, recall, FP-rate, F1-score

The simplest approach to handling these metrics is to compute them for each individual class, and then average over the classes

In this case, each individual class is in turn considered as the '*positive*' class while the others act as the '*negative*' clas

This does not account for class imbalance (some classes may have many more samples than others; more about this in future lecture)

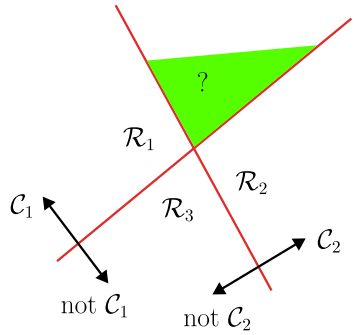
This can be handled by weighting the metric for each class by a value depending on the proportion of samples from that class.

So far, we have only seen how to extend binary least-square classification and binary logistic regression to the multi-class scenario. How about SVM? Unfortunately, there is no ideal

solution to handle multiple classes in SVM. Instead, one typically relies on one of the two solution described next.

First solution

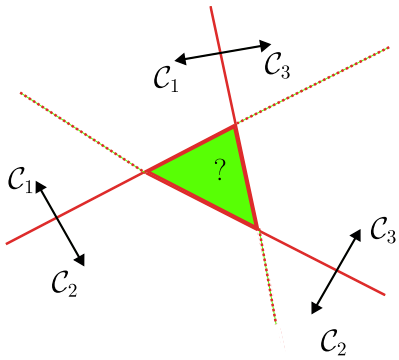
A solution is to use several binary classifiers. E.g., one-vs-rest: Classify each individual category vs all the other ones jointly



This however, leaves ambiguities (e.g., sample can be classified as being from both class 1 and class 2)

Second solution

We can only train it as one-vs-one: train one classifier for every pair of classes. With this technique we can output one class for each input:



This also may result in inconsistencies (e.g., a sample can be classified as being from class 1, class 2 and class 3)

Remark The green part here are elements which would be classified as more than one class (i.e., in the first solution, the green part would be classified as $\mathcal{R}_1$ **and** $\mathcal{R}_2$.

Comparing linear classifiers

We have now seen 3 different linear (multi-class) classifiers:

- least-square classification
- logistic regression
- SVM

In practice, one almost never uses the least-square one. It does not reflect the underlying classification task, but is useful from a pedagogical standpoint. But so which one is the best?

Comparison on the Iris dataset

UCI Iris dataset:

- 150 samples
- 4 attributes (petal and sepal length and width)
- 3 iris species

	Accuracy	training time	testing time
SVM, linear	88.0 %	0.006s	0.002s
Logistic regression (C = 1)	88.0 %	0.011s	0.0001s
Logistic regression (C = 1000)	92.00%	0.0010s	0.002s

The value C is a hyper-parameter encoding 'regularization' strength. We will talk about this in a future lecture; let's ignore it for now.

Comparing on the MNIST dataset

MNIST handwritten digit recognition dataset:

- 60'000 training samples, 10'000 test samples
- 28×28 images
- 10 digits, 0, ..., 9

	Accuracy	training time	testing time
SVM, linear	94.16 %	168.6950s	158.0101s
Logistic regression (C = 1)	91.47 %	272.1309s	0.0531s
Logistic regression (C = 1000)	91.23%	1807.0624s	0.0529s

Remark 13. I think here the C is not the same as the number of classes, our loss is defined using the pure cross-entropy:

$$R(W) = -\sum_{i=1}^{N}\sum_{k=1}^{C} y_i^{(k)} \ln \hat{y}_i^{(k)}$$

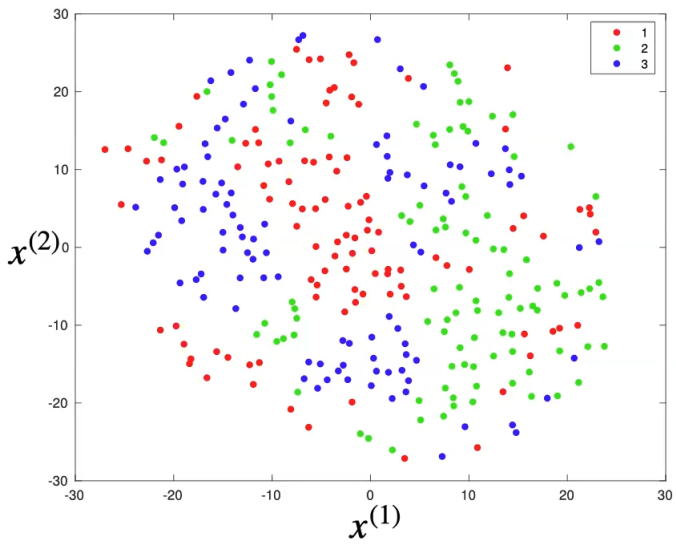
Where we added the regularization:

$$R(W) = \frac{1}{2C}\|\|W\|\|^2 - \sum_{i=1}^{N}\sum_{k=1}^{C} y_i^{(k)} \ln \hat{y}_i^{(k)}$$

And If I understand correctly why chatgpt said to me both C here are completely different.

1.6 k-nearest Neighbor

From the beginning of the course we only have used linear methods. the issue about them is that a lot of the data in our world cannot be described as a linear model, take for instance this dataset:



You cannot fit a linear model directly to this data.

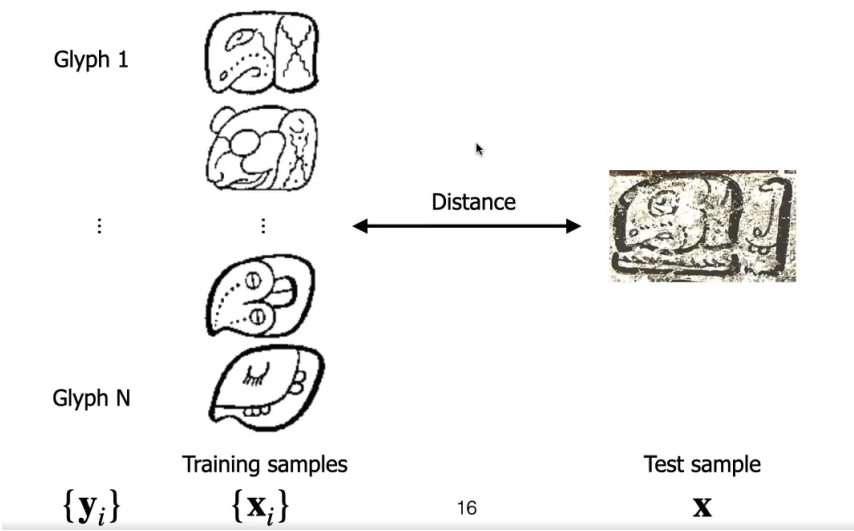
Intuition behind the nearest neighbor method

The intuition is that similar data samples have the same label. For instance if we take Maya glyph recognition (Hu et al., 2017) given a test sample, we look at the training sample that look a lot like the input sample and output the same ouput. This works with regression and classification.

1.6.1 NN: Algorithm

The algorithm can be computed as follow:

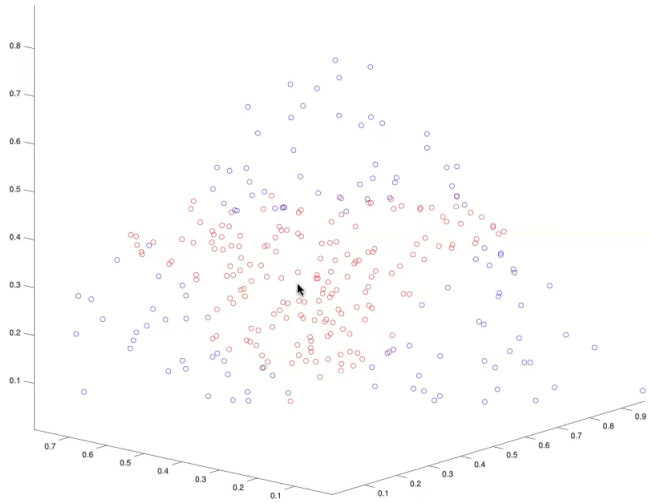
1. Compute the distance between the test sample x and all training samples $\{x_i\}$
2. Find the sample x_{NN} with minimum distance
3. Assin the corresponding label/value y_{NN} to the test sample



And this is our first **non-linear** algorithm!! In this case as in the linear model the results may change for different distance functions.

Example

Let us see a toy example with 3D histograms belonging to 2 classes (colors):



Histograms

Definition 22. A **histogram** in $\mathbb{R}^D$ is a D-dimensional vector such that:

$$x^{(d)} \in [0,1], \forall 1 \leq d \leq D$$
$$\sum_{d=1}^{D} x^{(d)} = 1$$

Given a vector $\mathbb{R}^D$ whose values are all non-negative (and at least one is stricly positive), one can obtain a histogram by dividing each value by the sum of values (normalize).

Example

Let the original vector in $\mathbb{R}^9$ be:

$$\widetilde{x} = \begin{pmatrix} 1 & 2 & 1 & 2 & 1 & 1 & 0 & 0 & 0 \end{pmatrix}^T$$

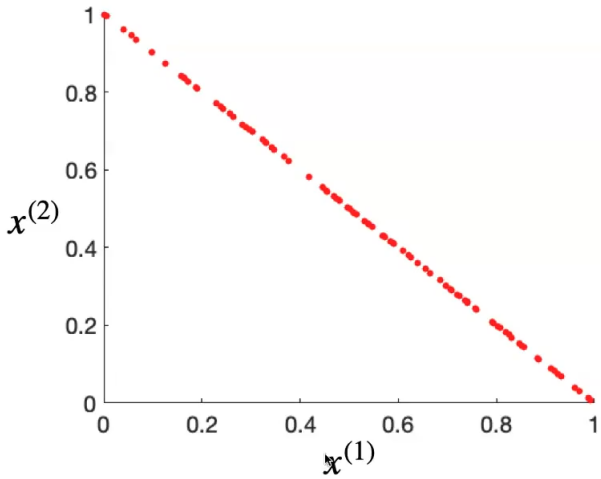
The sum of its value is:

$$\sum_{d=1}^{D} \widetilde{x}^{(d)} = 8$$

Then, the histogram can be computed as:

$$x = \begin{pmatrix} \frac{1}{8} & \frac{1}{4} & \frac{1}{8} & \frac{1}{4} & \frac{1}{8} & \frac{1}{8} & 0 & 0 & 0 \end{pmatrix}$$

intristingly, histograms live in a $D-1$ dimension space, for instance if we take $n = 100$ samples in dimension $D = 2$ then our histogram can be:



Back to NN

The default distance that we will use in our algorithm will be the euclidean distance:

$$d(x_i, x) = \sqrt{\sum_{d=1}^{D} \left(x_i^{(d)} - x^{(d)} \right)^2}$$

Another fairly common used distance is the **Chi-square** distance:

$$d(x_i, x) = \sqrt{\mathcal{X}^2(x_i, x)} = \sqrt{\sum_{d=1}^{D} \frac{\left(x_i^{(d)} - x^{(d)} \right)^2}{x_i^{(d)} + x^{(d)}}}$$

Another properties of the nn algorithm is that our results may be sensitive to outliers

- A point close to the outlier (blue circle in the center) will be missclassified
- Solution: Look at multiple neighbors instead of just one

This is the idea behind the k-nearest algorithm. Instead of taking only the nearest, we take k nearest.

1.6.2 k-nearest neighbors

The classification can be done as follow: **classification**

1. Compute the distance between the test sample x and all training samples $\{x_i\}$
2. Find the k samples $\{x_{NN1}, x_{NN2}, \ldots, x_{NNk}\}$ with minimum distances
3. Find the most common label among these k nearest neighbors (majority vote)
4. Assign the corresponding label y_{MV} to the test sample

If severel labels appear the same number of times among the $k - NN$, one strategy consists of taking the one whose samples have the smallest average distance to the test sample.

Regression

1. Compute the distance between the test sample x and all training samples $\{x_i\}$
2. Find the k samples $\{x_{NN1}, x_{NN2}, \ldots, x_{NNk}\}$ with minimum distances
3. Compute the value $\hat{y}$ for the test sample based on that of these $k - NN$

But how do we compute $\hat{y}$?

Two strategies

Intuitively we can either choose two things two in order to compute the label:

1. The number of label for each
2. The distance for each label

Using the average of the $k - NN$ values $\{x_{NNi}\}$:

$$\hat{y} = \frac{1}{k}\sum_{i=1}^{k} y_{NNi}$$

Using a weighted average of the $k - NN$ values, based on the inverse distances:

$$\hat{y} = \frac{1}{\sum_{i=1}^{k} w_{NNi}} \sum_{i=1}^{k} w_{NNi} y_{NNi}$$

Where

$$w_{NNi} = \frac{1}{d(x_{NNi}, x)}$$

Remark 14. To have a better visualization, we can use the demo here: http://vision.stanford.edu/teaching/cs231n-demos/knn/

k-nearest: Example

What about performance, if we take the MNIST classification (which has 10'000 data sampes) for each of our methods we have:

Method	Preprocessing	Error in %	Reference
K-nearest-neighbors, Euclidean (L_2)	none	5.0	LeCun et al.1998
K-nearest-neighbors, Euclidean (L_2)	none	3.09	Kennet Wilder. U. Chicago
K-nearest-neighbors, L_3	none	2.83	Kenneth Wilder. U. Chicago

If now we use preprocessing before using our algorithm defined above we can get:

Method	Preprocessing	Error in %	Reference
K-NN-neighbors, L_2	deskewing	2.4	LeCun et al.1998
K-NN-neighbors, L_2	deskewing, noise removal, blurring	1.80	Kennet U. Chicago
K-NNneighbors, L_3	deskewing, noise rem., blurring	1.73	Kenneth U. Chicago
K-NN-neighbors, L_3	deskewing, noise rem., blurring, 1 pixel shift	1.33	Kenneth U. Chicago
K-NN-neighbors, L_3	deskewing, noise rem., blurring, 2 pixel shift	1.22	Kenneth U. Chicago

Using even fancier preprocessing (shiftable edges, subsampling to 16x16 pixels etc...) we can get up to 0.52 % error.

Remark 15. Remember to put the face completion example

k-nearest neighbors: Propeties

There is no **learning** what we need is:

- a good data representation

- a distance function
- a given value k

This is referred to as a **non-parametric** learning method:

- No parameters to optimize during training
- k is said to be a **hyper-parameter**, set manually (or by validation, which we will discuss in future lecture)

As we have seen before, k-nn can be a very good a predicting but there is always a downside, it is **computationally expensive**. For **each test sample**, one need to compute the distances to **all training samples**. There is solution for this where we approcimate nearest neighbor search:

- Hashing
- kd-trees

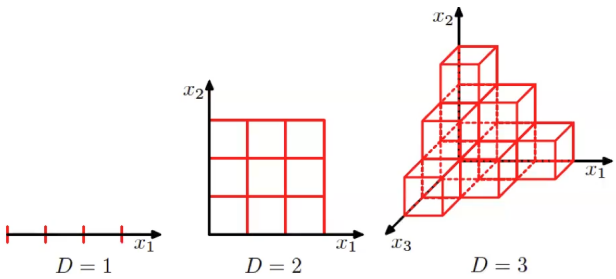
Curse of dimensionality

In the real world, data representations tend to be high dimensional, e.g., vectorizing an image yields a huge vector:

$$x_i \in \mathbb{R}^{536 \cdot 356 \cdot 3} = \mathbb{R}^{572448}$$

In such high dimension, all points tend to be far apart and we tends to have what we call curse of dimensionality (nummet mentionned??)

Let us divide the input space into regular cells: for dimensions $D = 1, 2, 3$ this gives



- The number of cells grows exponentially
- k-NN would then require an exponential growth of the data to cover the entire space

Handling high-dimensional data also make $k - NN$ memory inefficient, we need to store all the training data to compute distances.

Fortunately, real data often lies in much smaller subspaces, what we means by that is we don't necessarly need to span all space in order to make correct enough guess. Spanning a smaller spaces might be enough. In a few weeks, we will talk about how to find the low dimensional subspace of a given dataset.

1.6.3 Summary

Advantages

- Simple method
- Effective to handle nonlinear data
- Only requires defining one paramter (k)

Drawbacks

- The results depend on k
- Becomes expensive as N and/or D grow
- May be unreliable with a large D

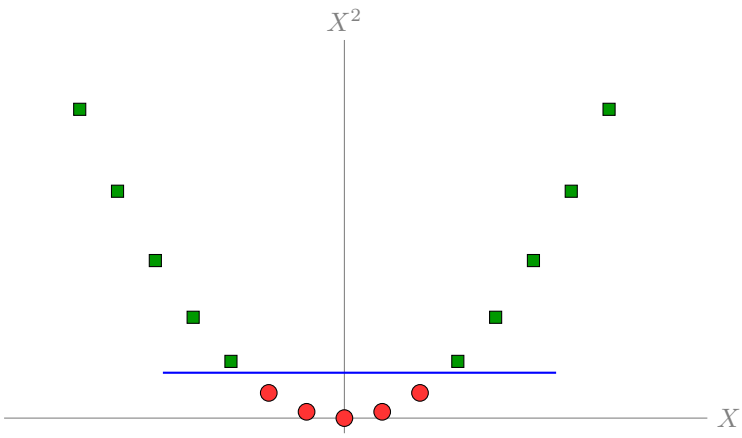
1.7 Nonlinear classification

The question we will try to answer now is: how can we hadle this data:

Now what if the red circle here are centered around 0. Then maybe by transforming our input data we will be able to handle it. Let us create a mapping:

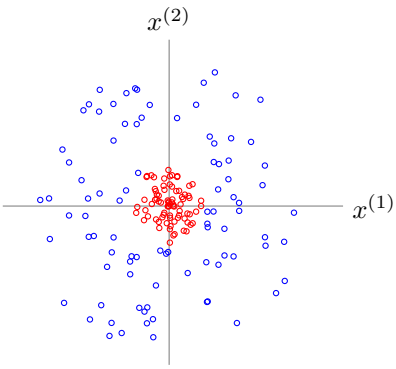
$$x \to [x, x^2]$$

Then if we mapped this new input:



In this new space we can now linearly seperate our data.

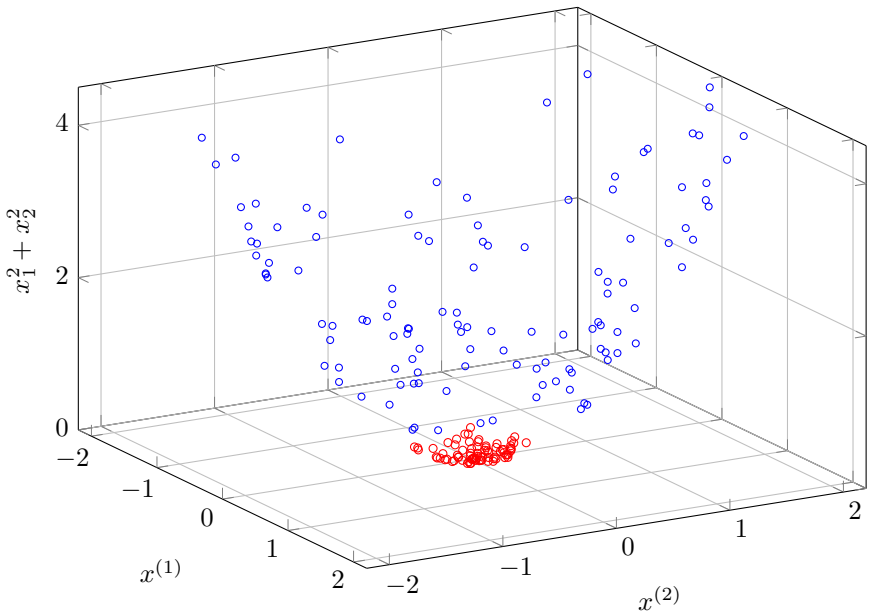
Now what if our data like this:



Intuitively this is the same case as the one before but in 2D, we could create a mapping for instance:

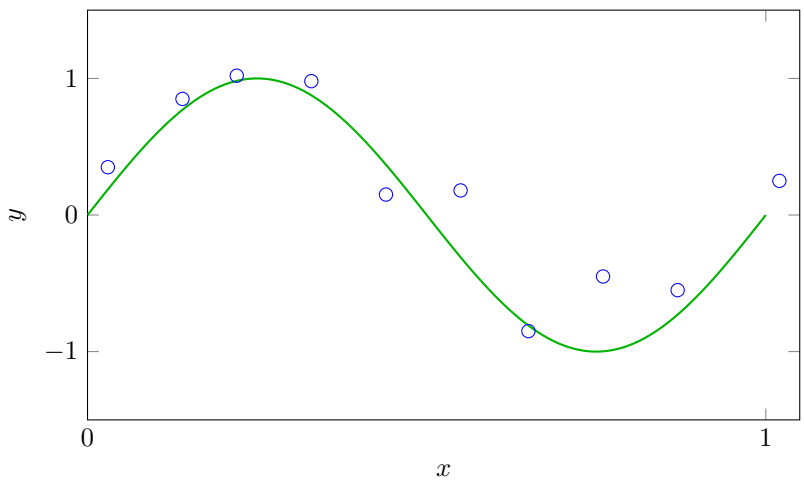
$$\begin{pmatrix} x^{(1)} \\ x^{(2)} \end{pmatrix} \to \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ \left(x^{(1)}\right)^2 \end{pmatrix}$$

Which should give the 'same' output as what we have done before. And this is pretty intuitive, we are mapping to the length of the circle as our data was bounded by a circle. And because our boundary is given as a circle, by 'mapping' our data into circle, then the boundary become linear.



1.7.1 Polynomial curve fitting

Given that we have training observations (blue circles, 1D input, 1D output) coming from the true green curve:



What we can do here is to find a polynomial that approximates the true curve using these observations.

Such a polynomial function of degree M can be written as:

$$y = w^{(0)} + w^{(1)}x + w^{(2)}x^2 + \cdots + w^{(M)}x^M = \sum_{j=0}^{M} w^{(j)}x^j$$

Where the $\{w^{(j)}\}$ are the coefficients of the different terms, but x^j represents x to the power j.

Polynomial feature expansion

In essence, in the three examples, we performed a change of variables:

- In the first case: $x \to \varphi(x) = \begin{pmatrix} x & x^2 \end{pmatrix}$

- In the second case: $x \to \varphi(x) = \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ \left(x^{(1)}\right)^2 + \left(x^{(2)}\right)^2 \end{pmatrix}$
- In the last (polynomial) case: $x \to \varphi(x) = \begin{pmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^M \end{pmatrix}$

In general, for D-dimensional inputs, we can obtain a large $\varphi(x)$ such that:

$$\varphi(x) = \begin{pmatrix} 1 \\ x^{(1)} \\ \vdots \\ x^{(D)} \\ \left(x^{(1)}\right)^2 \\ \vdots \\ \left(x^{(D)}\right)^2 \\ \vdots \\ \left(x^{(1)}\right)^M \\ \vdots \\ \left(x^{(D)}\right)^M \\ x^{(1)}x^{(2)} \\ \vdots \\ x^{(1)}x^{(D)} \\ \vdots \\ x^{(D-1)}x^{(D)} \\ \left(x^{(1)}\right)^2 x^{(2)} \\ \vdots \end{pmatrix}$$

An important thing to understand here is that we are just expanding our input to our usual linear model, nothing more. In essence, polynomial feature expansion uses a mapping from $x \in \mathbb{R}^D$ to another representation $\varphi(x) \in \mathbb{R}^F$, where $F \gg D$. But we can still use a linear model in this new space and write:

$$\hat{y}_i = w^T \varphi(x_i)$$

Where now $w \in \mathbb{R}^F$ (assuming that the 1 accounting for the bias has been incorporated in φ).

Training expanded features

For linear regression, we used the least-square loss:

$$\min_w \sum_{i=1}^N \left(w^T x_i - y_i\right)^2$$

With our expanded features, we can re-write this as:

$$\min_w \sum_{i=1}^N \left(w^T \varphi(x_i) - y_i\right)^2$$

Not that the w vector in the two equations above are different:

- In the first, $w \in \mathbb{R}^{D+1}$
- In the second, $w \in \mathbb{R}^{F}$

This new function is still convex, and its gradient is given by:

$$\nabla R = 2\sum_{i=1}^{N} \varphi(x_i)\Big(\varphi(x_i)^T w - y_i\Big)$$

Setting it to zero means that

$$\underbrace{\left(\sum_{i=i}^{N} \varphi(x_i)\varphi(x_i)^T\right)}_{F\times F} \underbrace{w^*}_{F\times 1} = \underbrace{\sum_{i=1}^{N} \varphi(x_i) y_i}_{F\times 1}$$

We then can group the transformed inputs $\{\varphi(x_i)\}$ and the output $\{y_i\}$ in a matrix and vector of the form

$$\Phi = \begin{pmatrix} \varphi(x_1)^T \\ \varphi(x_2)^T \\ vdd \\ \varphi(x_N)^T \end{pmatrix} \in \mathbb{R}^{N\times F}$$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^{N}$$

Then we have:

$$w^* = \left(\Phi^T\Phi\right)^{-1}\Phi^T y$$

Mapping to a higher-dimensional space

In all the examples considered before, the mapping φ increases the dimensionality of the input to the linear model:

- From 1 to 2 in the first toy example
- From 2 to 3 in the second one
- From 1 to $M+1$ in the polynomial fitting case

But why is higher dimension good?

Higher dimension give us more **flexibility**. There is a theorem called Cover's theorem which said that as you increase the dimension **non-linearly**, you increase the probability of making the data linearly seperable.

Note that it is important for the mapping φ to be **nonlinear**. A linear mapping would not result in a nonlinear model in the original space. To show this, let us consider the following linear mapping from a 2D input x to a 3D representation $\varphi(x)$:

$$\begin{pmatrix} x^{(1)} \\ x^{(2)} \end{pmatrix} \to \begin{pmatrix} x^{(1)} \\ x^{(2)} \\ x^{(1)} + x^{(2)} \end{pmatrix}$$

Then, a linear model applied to φ would have the form:

$$\begin{aligned} y &= w^{(0)} + w^{(1)}x^{(1)} + w^{(2)}x^{(2)} + w^{(3)}\Big(x^{(1)} + x^{(2)}\Big) \\ &= w^{(0)} + \Big(w^{(1)} + w^{(3)}\Big)x^{(1)} + \Big(w^{(2)} + w^{(3)}\Big)x^{(2)} \end{aligned}$$

Which is equivalent to:

$$y = w^{(0)} + \overline{w}^{(1)}x^{(1)} + \overline{w}^{(2)}x^{(2)}$$

Which implies that there is no nonlinear behavior.

Intuition Adding dimension which are linearly depedent from other is '*redundant*'. If φ was a linear function, then the job of this function could already been made by the model. The goal of adding a dimension was to do the job that the linear model couldn't do. Literaly any linear function can be done by the linear model (by definition).

Cover's theorem

Theorem 2 (Cover's theorem)**.**
A complex pattern-classification problem, cast in a high-dimensional space nonlinearly, is more likely to be linearly separable than in a low-dimensional space, provided that the space is not densely populated.

1.8 Overfitting, Cross-validation and Regularization

The goal of this section is to understand the notions of model complexity and overfitting. We will introduce several strategies to select a model that does not overfit and introduce the notion of regularization to prevent overfitting.

Model complexity

As we have moved from linear models to nonlinear ones, we have obtained increasing flexibility:

- We do not longer assume that the ouput linearly depends on the input
- We can handle classification problems in which the classes are nonlinearly separable

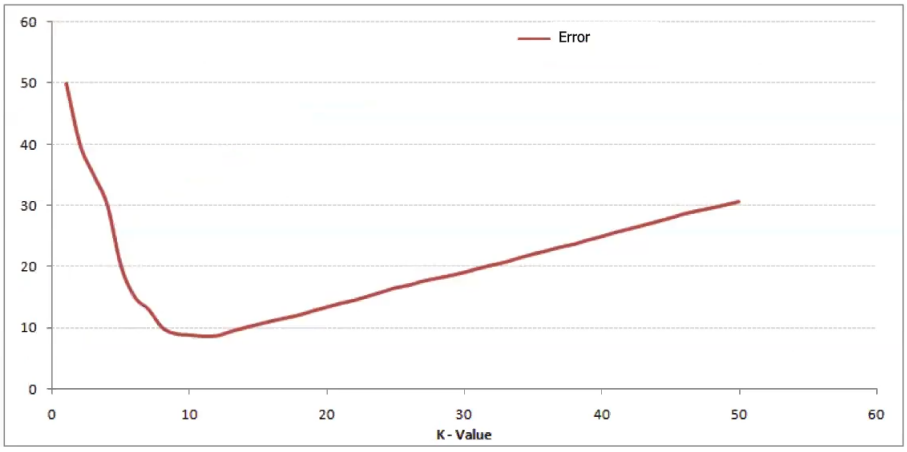
This flexibility, however, may sometimes make the model too complex for the task at hand. To illustrate this let us look at an example with k-NN:

Example

Using k-NN and some data for instance:
put images
The number of neighbors k influences a lot of our algorithm react for instance let us take some examples of values for k:
put another image here (both)o
As you can see by adding/removing k our algorithm our model differs. The question we are trying to answer is, what k is the best and first, what a low k do, what a high k do?

k-NN: Training vs test

Let's look at typical training and test curves (from T. Srivastava @ AnalyticsVidhya)



Test error

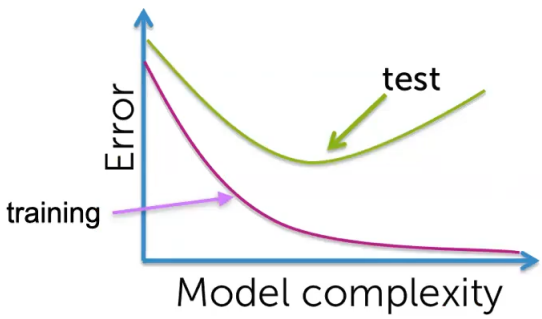
As we can see when having low k we are **overfitting**, meaning that copied our training data, but on on test data, the error will be high. And after passing 10 what we observed is that our model is actually performing worse than before even though we are incrementing k; our model is **underfitting**. In this case the error is large on both the training data and the test data. This is called **underfitting**.

Remark Tis behavior is **not** specific to k-NN. It can happen in most machine learning algorithm.

Remark 16. I find the example quite misleading. Here we see a low k as the overfitting case. As if the model is to easy then we are overfitting. But in general and as we will see later, the easier the model the more underfitting our model is doing. And it also works the other way around, the more our model is complex, the more our model is overfitting.

Overfitting vs Underfitting

Those are general phenomenons. And we can often links the error/fitting to the model complexity. By increasing the complexity we are also more prone to have better results on our training data, maybe too much which we will lead us to have a model that is overfitting.



Model Selection

The question is then:

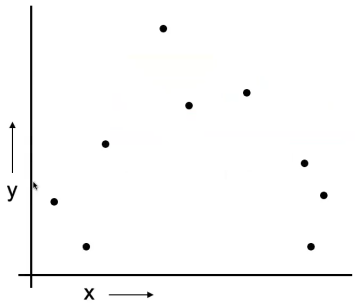
Since we don't have access to the test data, how do we choose the right complexity for the model?

Our solution is is **Cross-validation**

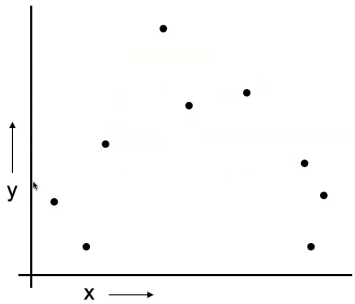
Remark The following material was adapted from Andrew Moore's cross-validation slides

Cross-validation: Toy example

Let us have a regression problem with a 1D input x and a 1D output y. We are given the following training observations:



We will consider 3 different models (of increasing complexity):

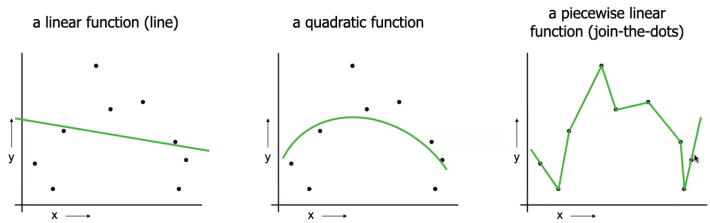


As our goal is to fit our data, why don't choose the one that best fits the data?

What we truly care about is how well the model is going to predict future data drawn from the same distribution. To do so, why not simulate data based on our training data.

Validation set method

We take our training data which we will split into two parts, one as the **training set**, and the other as a **validation set** (e.g., 30% of the data).



Advantages

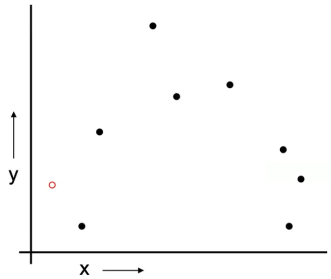
- Very simple
- We can choose the model based on the validation error

Leave-one-out cross-validation (LOOCV)

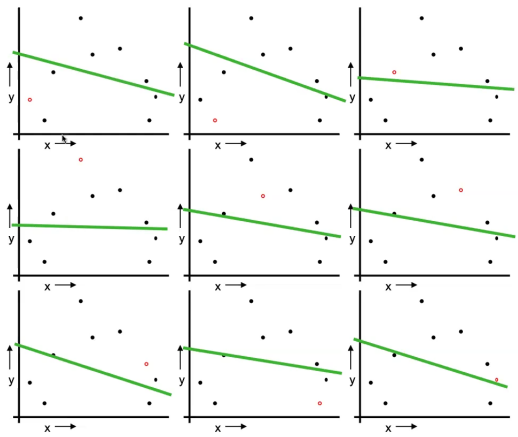
As an alternative method for the validation set method, we can use **LOOCV**: we will take one sample at a time and treat it as a validation sample, and we will loop through each sample as a validation sample.

For $i = 1, \ldots, N$

1. Let (x_i, y_i) be the ith sample
2. Temporarily remove (x_i, y_i) from the training set
3. Perform regression on the remaining $N - 1$ samples
4. Compute the error on (x_i, y_i)

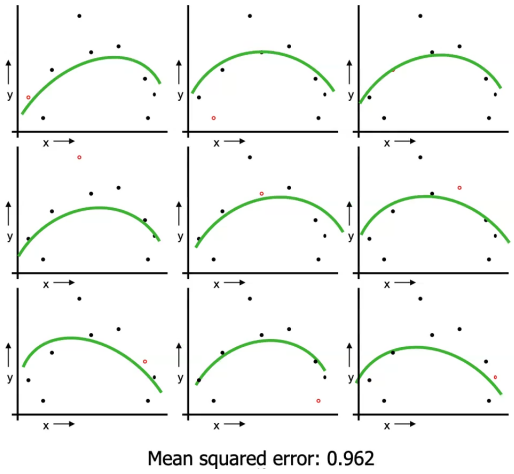


When you have done all points, report the average error. So if we take our 9 samples and we therfore fit 9 models, one for each points, we get the following curve (for a linear model)



For each of those model we have **one** test data which will gives us an error, given the average error for all models, we have 2.12.

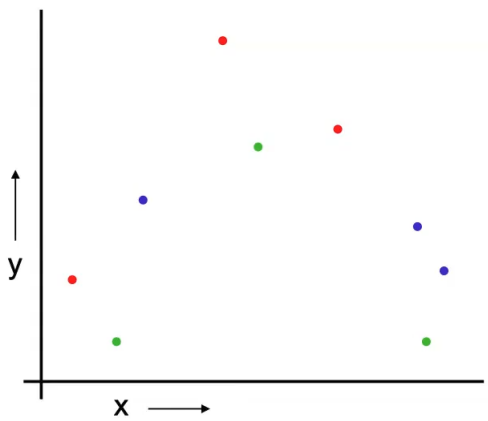
But is this the best model? Now we can check and be pretty sure about our metriics. For instance, let us use a squared model:



So as we can see, this type of cross-validation is pretty expensive, we have to train N times for **each model**! So which one is best?

	Downside	Upside
Validation-set	Unreliable estimate of future performance	Cheap
Leave-one-out	Expensive	Doesn't waste data

Both are a bit extreme, let us now look for something that is lying in the middle, ***k*-fold cross-validation**. First recall what were the issue about our two previous methods. The first one was that everything was fixed, the size of the test, the data that we used for training **and** for testing. The result of this is that the prediction of the future performance is unreliable (we could just be lucky with our test data). The issue with the second methods was that it was too expensive to compute because we needed to compute a new model **for each point**. But why each point, why don't we just randomly split the data set into k partitions (e.g., $k = 3$, shown with the different colors)



And now we do the same thing as what we did before but on partitions: we iterate through our partitions, we train our data in $\{\text{data set}\} \setminus \{\text{partitions}\}_i$. Then we compute the errors on the partitions_i. We then report the average error on all points.

	Downside	Upside
Validation-set	Unreliable estimate of future performance	Cheap
Leave-one-out	Expensive	Doesn't waste data
10-fold	Wastes 10% of the data. 10 times more expensive than Validation-set	Only wastes %. Only 10 times more expensive instead of N
3-fold	Wastier than 10-fold. Expensivier than validation-set	Slightly better than validation set
N-fold	Identival to Leave-one-out	

Remark 17. When we say "*Wastes 10%*" this doesn't mean that there is litteraly 10% of the data that go to the trash bin, what this means is that when **evaluating** the performance of our model, we loose does 10% for each fold. But not in the final training nor in the overall process of the cross-validation.

But ultimately there is not a best validation method. In practice one usually don't use Leave-one-out as it is too expensive. The k-fold is quite common when the model is not to expensive to train, but in the context of deep neural networks, almost everyone uses validation set.

Cross-validation

Cross-validation can help to:

- Select the best model among a small number of candidates
- Find the best value for one hyper-parameter (e.g. k in k-NN)
  - For this, one treats each value of the hyper-parameter as a separate model

However, it becomes impractival with too many candidates, for example, one cannot choose which combination of functions to use for features expansion among a very large set.

Now we have one tool in order to have a model that can generalize well. Let us now look at another way to prevent overfitting by penalizing model complexity via **regularization**. But before that let us go back to linear model.

Overfitting with a linear model

Because the linear model is simple, overfitting occurs fairly rarely. Nevertheless, if there are fewer training samples than input dimensions ($N \leq D+1$), then one can alwazs fit a hyperplan that passes exactly through the training samples.

Proof This can easily been check N points in $D+1$ dimensions. Our linear model will be given as $w^T x + b$. Let us simplify notation by absorbing the bias into the vector $\widetilde{x} = (x_1, \ldots, x_D, 1)$ then $\widetilde{w} \in \mathbb{R}^{D+1}$ and our model f is given as:

$$f(x) = \widetilde{w}^T \widetilde{x}$$

The input dimensions is $D + 1$ and the parameter vector is $w \in \mathbb{R}^{D+1}$. Given N training samples:

$$\left(x^{(1)}, y^{(1)}\right), \ldots, \left(x^{(N)}, y^{(N)}\right)$$

After the augmentation with have that each $\widetilde{x}^{(i)} \in \mathbb{R}^{D+1}$ Then what we atre saying is that we want our model to fit perfectly i.e.:

$$w^{(T)} \widetilde{x}^{(i)} = y^{(i)}, \quad \forall i = 1, \ldots, N$$

Now let us stack our vector $\widetilde{x}$ and defined $X \in \mathbb{R}^{N \times (D+1)}$, where each row is $\left(\widetilde{x}^{(i)}\right)^T$:

$$X = \begin{pmatrix} - & \left(\widetilde{x}^{(1)}\right)^T & - \\ - & \left(\widetilde{x}^{(2)}\right)^T & - \\ & \vdots & \\ - & \left(\widetilde{x}^{(N)}\right)^T & - \end{pmatrix}$$

This gives us the final system

$$Xw = y$$

This implies that we have a linear systems with N equations and $D+1$ unknows, this is solvable when $N \leq D+1$

Penalizin overfitting

THe standard way to penalize the complexity of a model is to add a regularizer to the empirical risk. This leads to a new training objective function of the form:

$$E(w) = R(w) + \lambda E_w(w)$$

Where $R(w)$ is the empirical risk and $E_w(w)$ is the regulariyer. λ is a hyper-parameter that defines the influence of the regularizer ($\lambda > 0$)

Regulariyed linear regression

One of the most common regularizers is the sum of squares of the weights (there is a probabilistic motivation behind it)

$$E_w(w) = w^T w$$

This lets us write regulariyed linear regression as

$$\min_w \sum_{i=1}^{N} \Big(\varphi(x_i)^T w - y_i\Big)^2 + \lambda w^T w$$

This remains a convex function (for $\lambda \geq 0$) and so we can still find the solution by zeroing out the gradient. But if you insists to compute the gradient…

$$\begin{aligned}\nabla E &= \nabla R + \nabla E_w \\ &= 2\sum_{i=1}^{N} \varphi(x_i)\Big(\varphi(x_i)^T w - y_i\Big) + 2\lambda w\end{aligned}$$

As we can see the only differences is the new $2\lambda w$, the other terms is the same as what we saw in earlier lectures.

Setting the gradient to zero yields

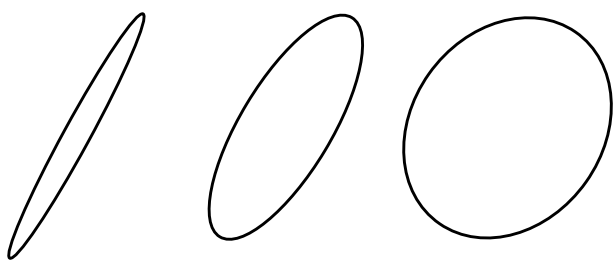
$$\left(\sum_{i=1}^{N} \varphi(x_i)\varphi(x_i)^T + \lambda I_F\right) w^* = \sum_{i=1}^{N} \varphi(x_i) y_i$$

Solution ultimately if we group the inputs in a matrix Φ and the output in a vector y, as before, this gives us the final solution:

$$w^* = \left(\Phi^T \Phi + \lambda I_F\right)^{-1} \Phi^T y$$

This also ensures that we can actually find a solution, that the matrix (the thing inside the parenthesis) can be inverted.

Remark For people who have done cs-328, this is the Tikhonov regulariyation technic that we have seen. In cs-328 the goal of regularization was to **improve the condition number** of a problem. The geometric interpretation was that the addition of an identiy matric causes the matrix to scale input vectors uniformly in different directions



small λ medium λ big λ

And this is the same thing as what we said above, the goal here is to improve the condition number $\implies$ having a matrix that is numericaly stable, that can be inverted

Note that the regularizer affects the optimal value of the parameters, but not the mathematical form of the prediction which remains the same.

Multi-output ridge regression

When dealing with multiple output dimesnions, the parameters are expressed in matrix form. We can write multi-output ridge regression as:

$$\min_W \sum_{i=1}^{N} \left\| W^T \varphi(x_i) - y_i \right\|^2 + \lambda \|W\|_F^2$$

Where $\|W\|_F^2$ is the square Frobenius norm, equivalent to the sum of the square of all values in the matrix W.

By zeroing out the gradient, we obtain the solution:

$$W^* = \left(\Phi^T \Phi + \lambda I_F\right)^{-1} \Phi^T Y$$

Regularization and logistic regression

Regularization may also be added to the cross-entropy loss used to train the logistic regression model.

- With a convex regularizer, such as the sum of squares, the overall regularized risk remains convex
- The scikit-learn implementation of logistic regression has such an L^2 regularizer by default

Finding the right regularization strength

Note that the use of a regularizer leads to another hyper-parameter for the model, λ. And this parameter cannot be optimize, (the solution for it is $\lambda = 0$ which is not what we wanted and the reason on why we have introduced it in the first place). But if we cannot optimize it, how should we set its value? IDK

1.9 Kernel methods

The goal of today's lecture is to introduce the notion of kernel function and the kernel trick, to do so, we need to go back to the linear model:

$$y = w^{(0)} + w^{(1)} x^{(1)} + \cdots + w^{(D)} x^{(D)} = w^T \begin{pmatrix} 1 \\ x^{(1)} \\ x^{(2)} \\ \vdots \\ x^{(D)} \end{pmatrix}$$

From this model, we import a non linear machine algorithm by first doing a non linear function and **then** apply our linear model to it, for instance the polynomial feature expansion:

$$\varphi(x) = \begin{pmatrix} 1 \\ x^{(1)} \\ \vdots \\ x^{(D)} \\ \left(x^{(1)}\right)^2 \\ \vdots \\ \left(x^{(D)}\right)^2 \\ \vdots \\ \left(x^{(1)}\right)^M \\ \vdots \\ \left(x^{(D)}\right)^M \\ x^{(1)}x^{(2)} \\ \vdots \\ x^{(1)}x^{(D)} \\ \vdots \\ x^{(D-1)}x^{(D)} \\ \left(x^{(1)}\right)^2 x^{(2)} \\ \vdots \end{pmatrix}$$

In essence, polynomial feature expansion uses a mapping from $x \in \mathbb{R}^D$ to another representation $\varphi(x) \in \mathbb{R}^F$, where $F \gg D$

We can the use a linear model in this new space and write:

$$\hat{y}_i = w^T \varphi(x_i)$$

Given training data, we can the find the optimal parameters via standard linear regression

$$\min_w \sum_{i=1}^{N} \left(w^T \varphi(x_i) - y_i\right)^2$$

When having more dimension, we can always transformed our inputs $\{\varphi(x_i)\}$ and the output $\{y_i\}$ in a matrix form and vector of the form:

$$\Phi = \begin{pmatrix} \varphi(x_1)^T \\ \varphi(x_2)^T \\ \vdots \\ \varphi(x_N)^T \end{pmatrix} \in \mathbb{R}^{N \times F} \qquad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix}$$

Then we have $w^* = \left(\Phi^T \Phi\right)^{-1} \Phi^T y$

Remark 18. Maybe a little remark that I want to do here about the pseudo inverse and its definition. I think it is a bit misleading to **define** the pseudoinverse as $\left(\Phi^T \Phi\right)^{-1} \Phi^T y$, a "*more intuitive way*" would maybe be from the SVD decomposition and how does an inverse (from a non square matrix behaves). We can kind of see it as a generalisation of the inverse, which means if we have n data pointer and n variable then we can create the inverse without any issue $\implies$ We are *overfitting* our model. But if we have too much data point then the pseudoinverse still have to represent something meaningful in terms of the solution of the equation $\implies$ it gives the minimum-norm solution. On the other case if we have a tall matrix $\implies$ it gives the least squares solution.

The conclusion is that we compute the pseudoinverse using SVD and this gives us a generalization of the inverse of a matrix.

But we don't have to limit ourself to polynomials, we could add sine cosine, exponential, etc... But how do we choose which functions to use? The solution to this is **Kernels**

1.10 Kernel function

The solution to many machine learning algorithms ends up relying on a dot product between inputs, i.e. $x_i^T x_j$. $x_i^T x_j$ is proportional to $\cos(x_i, x_j)$ this encodes a notion of similarity between two samples

When doing feature expansion, we then have dot products of the form $\varphi(x_i)^T \varphi(x_j)$, which also encodes a similarity between the two samples, but different from the original one

Thus, instead of explicitly defining the mapping $\varphi()$, one can define a similarity function $k(x_i, x_j)$. This is called a **kernel** function. It corresponds to some mapping φ, i.e., $k(x_i, x_j) = \varphi(x_i)^T \varphi(x_j)$, but this φ may be unknown.

Example

One example of commonly used kernel function is the polynomial kernel

$$k(x_i, x_j) = \left(x_i^T x_j + c\right)^d$$

It has two hyper-parameters, the bias c and the degree of the polynomial d (often set to 2)

For such kernels, the corresponding mappings φ are know, for example, with a 2D input, for $c = 1$ and $d = 2$ we have

$$\varphi(x) = \left[\left(x^{(1)}\right)^2, \left(c^{(2)}\right)^2, \sqrt{2}x^{(1)}x^{(2)}, \sqrt{2}x^{(1)}, \sqrt{2}x^{(2)}, 1\right]$$

Given our mapping the dot product between two samples then is:

$$\begin{aligned}\varphi(x_i)^T \varphi(x_j) &= \left(x_i^{(1)}\right)^2 \left(x_j^{(1)}\right)^2 + \left(x_i^{(2)}\right)^2 \left(x_j^{(2)}\right)^2 + 2x_i^{(1)}x_i^{(2)}x_j^{(1)}x_j^{(2)} + 2x_i^{(1)}x_j^{(1)} + 2x_i^{(2)}x_j^{(2)} + 1 \\ &= \left(x_i^{(1)}x_j^{(1)} + x_i^{(2)}x_j^{(2)} + 1\right)^2\end{aligned}$$

This corresponds to the quadratic kernel:

$$k(x_i, x_j) = \left(x_i^T x_j + 1\right)^2$$

Example II

Another common kernel is the Gaussian (or Radial Basis Function (RBF)) kernel

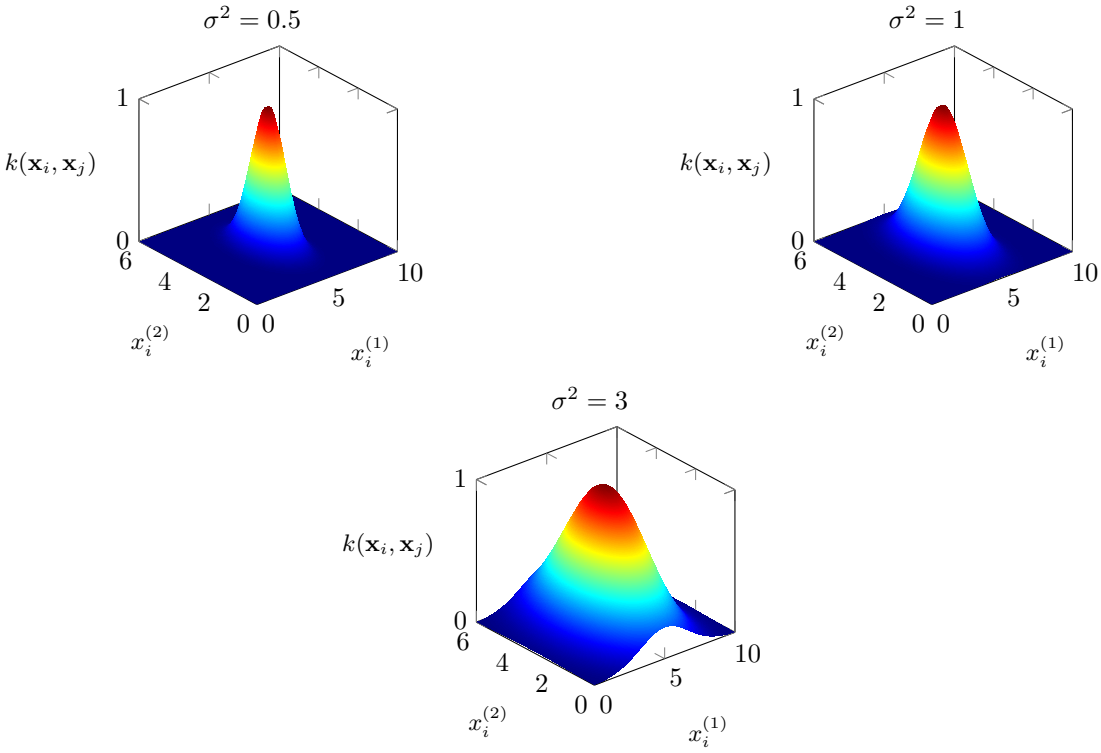
$$k(x_I, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

It has one hyper-parameter which is the *bandwidth* σ

For this kernel, there is no explicit corresponding mapping φ

Remark In theory, this kernel corresponds to a mapping to an infinite dimensional space

With 2D inputs, we can visualize the effect of σ, Let one sample be fixed as $x_j = [5, 3]^T$, and very x_i



Kernel trick

Since a kernel function corresponds to a similarity, as a dot product, one can replace any dot product between two samples in the solution of an machine learning algorithm with a kernel function. This is applicable to many algorithms, and is referred to as **kernelizing** an algorithm

Let us now look at how this works for linear regression

1.10.1 Kernelization

Notation

As mentioned before, the kernel function computes a similarity between two samples. It can be written as $k(x_i, x_j)$

In what follows, with a small abuse of notation, we will also use the kernel function to encode the similarity between one sample and all training samples, represented vie the matrix X. This will give:

$$k(X, x_i) = \begin{pmatrix} k(x_1, x_i) \\ k(x_2, x_i) \\ \vdots \\ k(x_N, x_i) \end{pmatrix} \in \mathbb{R}^N$$

Furthermore, we will also use the notion of kernel matrix, in which each entry (i, j) contains the kernel function evaluated between training sample i and training sample

j. This matrix is squared and can be written as

$$K = \begin{pmatrix} k(x_1, x_2) & k(x_1, x_2) & \cdots & k(x_1, x_N) \\ k(x_2, x_1) & k(x_2, x_2) & \cdots & k(x_2, x_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(x_N, x_1) & k(x_N, x_2) & \cdots & k(x_N, x_N) \end{pmatrix} \in \mathbb{R}^{N \times N}$$

Kernel regression

As seen before, the solution to linear regression is given by:

$$w^* = \left(\Phi^T \Phi\right)^{-1} \Phi^T y$$

At first sight, it may seem that $\Phi^T \Phi$ contains the dot products that we want to replace with a kernel function. However, $\Phi^T \Phi$ is an $F \times F$ matrix encoding dot products between the different feature dimensions, not the different samples. A kernel matrix, containing the dot products between every pair of training samples would be of size $N \times N$ and correspond to $\Phi \Phi^T$. Therefore we need another solution, the Representer theorem

Representer theorem

Theorem 3.
The minimizer of a regularized empirical risk function can be represented as a linear combination of the points in the high dimensional space. That is, we can write

$$w^* = \sum_{i=1}^{N} \alpha_i^* \varphi(x_i) = \Phi^T \alpha^*$$

In general if I am trying to find the minimizer, then I can represent the vector w^* as a linear combination of my training samples, with some coefficients α_i^*. Therefore $\alpha^* \in \mathbb{R}^N$

Kernel regression

So instead of searching for a minimizer of the empirical risk in terms of w, we search for one in terms of new variables $\{\alpha_i\}$

That is, the empirical risk

$$R(w) = \sum_{i=1}^{N} \left(w^T \varphi(x_i) - y_i\right)^2$$

What we will do, is to use the Representer theorem in order to rewrite our function with variables α

$$R(\alpha) \sum_{i=1}^{N} \left(\alpha^T \Phi \varphi(x_i) - y_i\right)^2$$

But now we have the dot products that we needed before. This implies that we can rewrite using the kernel

$$= \sum_{i=1}^{N} \left(\alpha^T k(X, x_i) - y_i\right)^2$$

The gradient of the risk with regards to α is given by

$$\nabla R(\alpha) = 2\sum_{i=1}^{N} k(X, x_i)\Big(k(X, x_i)^T\alpha - y_i\Big)$$

Following the same reasoning as for linear regression, we can write this matrix form as

$$\nabla_\alpha R(\alpha) = 2KK\alpha - 2Ky$$

Setting the gradient to 0 yields:

$$2KK\alpha^* = 2Ky \iff K\alpha^* = y$$

Which gives the solution

$$\alpha^* = (K)^{-1}y$$

If we put this solution back in the definition of w^*, we have:

$$\begin{aligned} w^* &= \Phi^T\alpha^* \\ &= \Phi^T(K)^{-1}y \end{aligned}$$

This solution still depends on Φ, which we argued could be unknown? But do we care about computing the weights? In practice, this is not a problem, because our goal is to make predictions for new samples, we do not really care about having access to w^*

For a new sample x, the prediction is given by

$$\begin{aligned} \hat{y} &= (w^*)^T\varphi(x) = y^T(K)^{-1}\Phi\varphi(x) \\ &= y^T(K)^{-1}k(X, x) \end{aligned}$$

Not that the quantity $y^T(K)^{-1}$ only depends on the training data, and thus can be precomputed (i.., does not need to be recomputed for every test sample)

Exercise: Kernel regression

You are given the following 4 training samples (x_i, y_i)

$$\left(\begin{pmatrix}1\\1\end{pmatrix}, 0\right) \quad \left(\begin{pmatrix}0\\0\end{pmatrix}, 2\right)\left(\begin{pmatrix}2\\0\end{pmatrix}, 0\right) \quad \left(\begin{pmatrix}2\\2\end{pmatrix}, 2\right)$$

The task is to compute the kernel matrix corresponding to a polynomial kernel of degree 2 with a bias 1. (If I have time I will do it here)

Recap: Ridge regression

To penalize overfitting, one can use a regularizer such as the sum of squares of the weights

$$E_w(w) = w^Tw$$

Adding this to the linear regression empirical risk yields the ridge regression problem

$$\min_w \sum_{i=1}^{N}\Big(\varphi(x_i)^Tw - y_i\Big)^2 + \lambda w^Tw$$

What we would want here is to also translate it to a kernel. To do so we can use the Representer theorem, this way, we can rewrite it as:

$$\begin{aligned} E(\alpha) &= \sum_{i=1}^{N} \left(\alpha^T \Phi \varphi(x_i) - y_i\right)^2 + \lambda \alpha^T \Phi \Phi^T \alpha \\ &= \sum_{i=1}^{N} \left(\alpha^T k(X, x_i) - y_i\right)^2 + \lambda \alpha^T K \alpha \end{aligned}$$

Following the same reasoning as without regularizer, we can write the gradient of the risk in matrix form as

$$\nabla_\alpha E(\alpha) = 2K(K + \lambda I_N)\alpha - 2Ky$$

Setting it to 0 gives us the solution:

$$\alpha^* = (K + \lambda I_N)^{-1} y$$

Given α^*, one can then express W^* mathematically as in the case without regularization, and eventually use this formulation to make predictions for the test samples

The kernel of the ridge regression strongly depend on the choice of kernel and of the hyper-parameter values for a given kernel. How can you identify a good kernel and good parameter values??

We know how to choose them, using **cross-validation**.

Remark 19. We can do the same thing on ridge regression with multiple outputs (the exact same process), the prediction vector for a test sample is then given as

$$\begin{aligned} \hat{y} &= (W^*)^T \varphi(x) = Y^T (K + \lambda I)^{-1} \Phi \varphi(x) \\ &= Y^T (K + \lambda I)^{-1} k(X, x) \end{aligned}$$

Kernelization

The idea of kernelizing an algorithm does not apply only to linear regression

- It can also be used to obtain a nonlinear version of SVM (via the so-called dual formulation of SVM)
- In a few weeks, we will also do this for dimensionality reduction
- Not that logistic regression can also be kernelized although this is less commonly done

Chapter 2

Supervised Linear ML

Chapter 3

Unsupervised Linear ML